

Describing Syntax

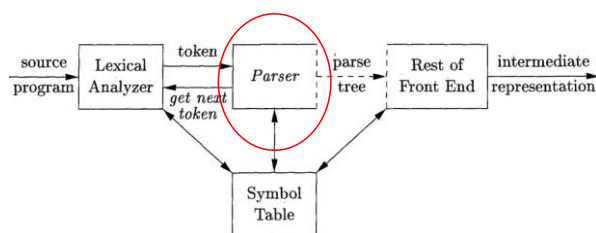
- The syntax of programming language constructs can be specified by **context-free grammars** or BNF (Backus-Naur Form) notation
 - A grammar gives a precise, yet easy-to-understand, **syntactic specification** of a programming language
 - For certain grammars, we can **automatically construct an efficient parser**
 - A properly designed grammar **defines the structure of a language** and helps translate source programs into correct object code and detect errors
 - A grammar allows a language to be **evolved or developed iteratively**, by adding new constructs to perform new tasks

编程语言结构的语法可以通过上下文无关文法或 BNF（巴科斯范式）表示法来指定，在 2.2 节中介绍。文法为语言设计者和编译器编写者提供了巨大的好处。

- 文法给出了一个程序设计语言的精确且易于理解的语法规范。
- 对于某些类型的文法，我们可以自动构造一个有效的语法分析器，它能够确定一个源程序的语法结构。语法分析器的构造过程可以揭示语法的二义性，同时还可能发现一些容易在语言的初始设计阶段被忽略的问题。
- 一个正确设计的文法给出了一个语言的结构。该结构有助于把源程序翻译成正确的目标代码，也有助于检测错误。
- 一个文法支持逐步加入可以完成新任务的新语言构造从而迭代地演化和开发语言。如果对语言的实现遵循语言的文法结构，那么在实现中加入这些新构造的工作就变得更加容易。

The Role of the Parser

- The parser obtains a string of tokens from the lexical analyzer and verifies that the string of token names can be generated by the grammar for the source language
- Report syntax errors in an intelligent fashion
- For well-formed programs, the parser constructs a parse tree
 - The parse tree need not be constructed explicitly



语法分析器的角色

语法分析器从词法分析器获得 token 串，并验证 token 名称是否可以由源语言的语法生成

以智能的方式报告语法错误

对于格式良好的程序，语法分析器构建一个语法分析树

语法分析树无需显式构建

Classification of Parsers

- **Universal parsers (通用语法分析器)**
 - Some methods (e.g., Earley's algorithm¹) can parse any grammar
 - However, they are too inefficient to be used in practice
- **Top-down parsers (自顶向下语法分析器)**
 - Construct parse trees from the top (root) to the bottom (leaves)
- **Bottom-up parsers (自底向上语法分析器)**
 - Construct parse trees from the bottom (leaves) to the top (root)

Note: Top-down and bottom-up parsing both scan the input from left to right, one symbol at a time. They work only for certain grammars, which are expressive enough.

¹ <http://loup-vaillant.fr/tutorials/earley-parsing/>

语法分析器的分类

- 通用语法分析器
 - 一些方法（例如，Earley 算法 1）可以解析任何语法
 - 然而，它们效率太低，无法在实践中使用
- 自顶向下语法分析器
 - 构建从顶部（根）到底部（叶）的语法分析树
- 自底向上语法分析器
 - 从底部（叶子）到顶部（根）构建语法分析树

注意：自上而下和自下而上的解析都从左到右扫描输入，一次扫描一个符号。它们只适用于某些语法，这些语法具有足够的表达能力。

Context-Free Grammar (上下文无关文法)

- A context-free grammar (CFG) consists of four parts:
 - **Terminals (终结符号):** Basic symbols from which strings are formed (token names)
 - **Nonterminals (非终结符号):** Syntactic variables that denote sets of strings
 - Usually correspond to a language construct, such as *stmt* (statements)
 - One nonterminal is distinguished as the **start symbol (开始符号)**
 - The set of strings denoted by the start symbol is the language generated by the CFG
 - **Productions (产生式):** Specify the manner in which the terminals and nonterminals can be combined to form strings
 - **Format:** head (left side) $\rightarrow$ body (right side)
 - The **head** is a nonterminal; the **body** consists of zero or more terminals/nonterminals
 - **Example:** *expression* $\rightarrow$ *expression* + *term*

上下文无关语法（CFG）由四个部分组成：

- 终结符号：形成字符串的基本符号（token 名称）
- 非终结符号：表示字符串集的语法变量 通常对应于一个语言结构，如 *stmt* (statements)
- 我们指定一个非终结符作为开始符号 由开始符号表示的字符串集是 CFG 生成的语言
- 产生式：指定终结符号和非终结符号组合形成字符串的方式
- 格式：head（左侧） $\rightarrow$ body（右侧）
- head 是非终结符号；body 由零个或多个终结符号或非终结符号组成

例如：*expression* $\rightarrow$ *expression* + *term*

CFG Example

- The grammar below defines simple arithmetic expressions
 - **Terminal symbols:** `id`, `+`, `-`, `*`, `/`, `(`, `)`
 - **Nonterminals:** `expression`, `term`, `factor`
 - **Start symbol:** `expression`
 - **Productions:**
 - `expression` $\rightarrow$ `expression` `+` `term`
 - `expression` $\rightarrow$ `expression` `-` `term`
 - `expression` $\rightarrow$ `term`
 - `term` $\rightarrow$ `term` `*` `factor`
 - `term` $\rightarrow$ `term` `/` `factor`
 - `term` $\rightarrow$ `factor`
 - `factor` $\rightarrow$ `(` `expression` `)`
 - `factor` $\rightarrow$ `id`

CFG 示例

下面的语法定义了简单的算术表达式

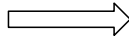
终结符: `id`, `+`, `-`, `*`, `/`, `(`, `)`

非终结符: `expression`, `term`, `factor`

开始符: `expression`

Notational Simplification

```
expression → expression + term  
expression → expression - term  
expression → term  
term → term * factor  
term → term / factor  
term → factor  
factor → ( expression )  
factor → id
```



```
E → E + T | E - T | T  
T → T * F | T / F | F  
F → ( E ) | id
```

- **|** is a **meta symbol** to specify alternatives
- **(** and **)** are not meta symbols, they are terminal symbols

符号简化

“|”是指定备选方案的元符号

“(”和”)”不是元符号，而是终结符号

Derivations

- **Derivation (推导):** Starting with the start symbol, nonterminals are rewritten using productions until only terminals remain
- Example:
 - CFG: $E \rightarrow - E \mid E + E \mid E * E \mid (E) \mid \text{id}$
 - A derivation (a sequence of rewrites) of $-(\text{id})$ from E
 - $E \Rightarrow -E \Rightarrow -(E) \Rightarrow -(\text{id})$

推导: 从开始符号开始, 使用产生式重写非终结符, 直到只剩下终结符为止

示例:

从 E 导出 $-(\text{id})$ (一系列重写)

Notations

- $\Rightarrow$ means “derives in one step”
- $\overset{*}{\Rightarrow}$ means “derives in zero or more steps”
 - $\alpha \overset{*}{\Rightarrow} \alpha$ holds for any string α
 - If $\alpha \overset{*}{\Rightarrow} \beta$ and $\beta \Rightarrow \gamma$, then $\alpha \overset{*}{\Rightarrow} \gamma$
 - Example: $E \Rightarrow -E \Rightarrow -(E) \Rightarrow -(\mathbf{id})$ can be written as $E \overset{*}{\Rightarrow} -(\mathbf{id})$
- $\overset{+}{\Rightarrow}$ means “derives in one or more steps”

符号

$\Rightarrow$ 意思是单步推导

$\Rightarrow^*$ 意思是“以零个或多个步骤推导” $\alpha \Rightarrow^* \alpha$ 适用于任何字符串 α

$\Rightarrow^+$ 意思是“以一个或多个步骤推导”

Terminologies

- If $S \Rightarrow^* \alpha$, where S is the start symbol of a grammar G , we say that α is *sentential form* of G (文法的句型)
 - May contain both terminals and nonterminals, and may be empty
 - **Example:** $E \Rightarrow -E \Rightarrow -(E) \Rightarrow -(E + E) \Rightarrow -(\mathbf{id} + E) \Rightarrow -(\mathbf{id} + \mathbf{id})$, here all strings of grammar symbols are sentential forms
- A *sentence* (句子) of G is a sentential form with no nonterminals
 - In the above example, only the last string $-(\mathbf{id} + \mathbf{id})$ is a sentence
- The *language generated* by a grammar is its set of sentences

术语

- 如果 $S \Rightarrow^* \alpha$, 其中 S 是语法 G 的开始符号, 我们说 α 是 G 文法的句型
 - 文法的句型可以同时包含终结符号和非终结符号, 并且可以为空
 - 示例 $E \Rightarrow -E \Rightarrow -(E) \Rightarrow -(E + E) \Rightarrow -(\mathbf{id} + E) \Rightarrow -(\mathbf{id} + \mathbf{id})$ 这里所有的语法符号串都是文法的句型
- 一个 G 的句子是一个没有非终结符的文法句型
 - 在上面的例子中, 只有最后一个字符串 $-(\mathbf{id} + \mathbf{id})$ 是一个句子
- 语法产生的语言是它产生句子的集合

Leftmost/Rightmost Derivations

- At each step of a derivation, we need to choose which nonterminal to replace
- In **leftmost derivations (最左推导)**, the leftmost nonterminal in each sentential form is always chosen to be replaced

$$\blacksquare E \xRightarrow{lm} - E \xRightarrow{lm} - (E) \xRightarrow{lm} - (E + E) \xRightarrow{lm} - (\mathbf{id} + E) \xRightarrow{lm} - (\mathbf{id} + \mathbf{id})$$

- In **rightmost derivations (最右推导)**, the rightmost nonterminal is always chosen to be replaced

$$\blacksquare E \xRightarrow{rm} - E \xRightarrow{rm} - (E) \xRightarrow{rm} - (E + E) \xRightarrow{rm} - (E + \mathbf{id}) \xRightarrow{rm} - (\mathbf{id} + \mathbf{id})$$

最左/最右推导

在推导的每一步，我们都需要选择要替换的非终结符

在最左推导中,总是选择每个句型中最左边的非终结符来替换

在最右推导中,总是选择最右边的非终结符来替换

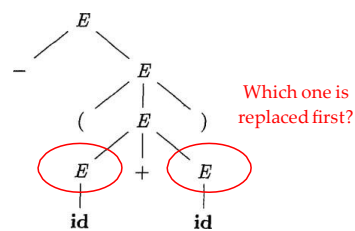
Parse Trees (语法分析树)

- A **parse tree** is a graphical representation of a derivation
 - The **root node** (根结点) is the start symbol of the grammar
 - Each **leaf node** (叶子结点) is labeled by a terminal symbol or ϵ
 - Each **interior node** (内部结点) is labeled by a nonterminal symbol
 - Each interior node represents the application of a production
 - The interior node is labeled with the nonterminal in the head of the production; the children nodes are labeled, from left to right, by the symbols in the body of the production

CFG: $E \rightarrow - E \mid E + E \mid E * E \mid (E) \mid \text{id}$

$E \xRightarrow{lm} - E \xRightarrow{lm} - (E) \xRightarrow{lm} - (E + E) \xRightarrow{lm} - (\text{id} + E) \xRightarrow{lm} - (\text{id} + \text{id})$

$E \xRightarrow{rm} - E \xRightarrow{rm} - (E) \xRightarrow{rm} - (E + E) \xRightarrow{rm} - (E + \text{id}) \xRightarrow{rm} - (\text{id} + \text{id})$



语法分析树是一种推导的图形表示

- 根结点是文法的开始符号
- 每个叶子结点由终结符或 ϵ 标记
- 每个内部结点由非终结符标记
- 每个内部结点代表产生式的应用
 - 内部节点用产生式的头部中的非终结符进行标记;
 - 子节点由产生式主体中的符号从左到右进行标记

问题：语法分析树中红色虚线圈起来的 E 哪一个会先被替换？

答案：在最左推导中，左孩子先被替换；在最右推导中，右孩子先被替换。

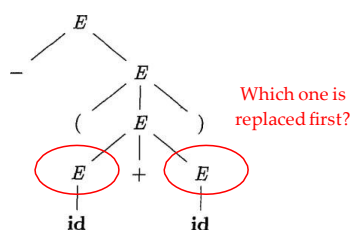
Parse Trees (语法分析树) Cont.

- The leaves, from left to right, constitute a **sentential form** of the grammar, which is called the *yield* or *frontier* of the tree
- There is a **many-to-one** relationship between **derivations** and **parse trees**
- There is a **one-to-one** relationship between **leftmost/rightmost derivations** and **parse trees**

CFG: $E \rightarrow - E \mid E + E \mid E * E \mid (E) \mid \text{id}$

$E \xRightarrow{lm} - E \xRightarrow{lm} - (E) \xRightarrow{lm} - (E + E) \xRightarrow{lm} - (\text{id} + E) \xRightarrow{lm} - (\text{id} + \text{id})$

$E \xRightarrow{rm} - E \xRightarrow{rm} - (E) \xRightarrow{rm} - (E + E) \xRightarrow{rm} - (E + \text{id}) \xRightarrow{rm} - (\text{id} + \text{id})$



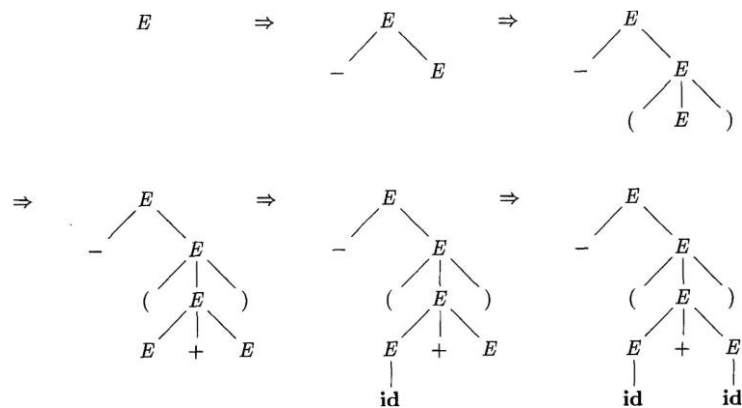
从左到右，叶子结点构成了语法的一种句型，称为树的产出或边缘

推导和语法分析树之间存在多对一关系

最左/最右推导和语法分析树之间存在一对一关系

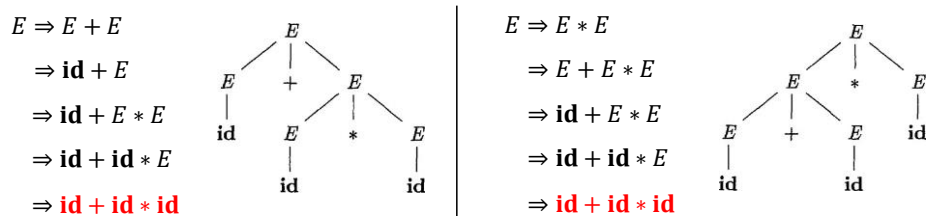
Constructing Parse Trees (Example)

Derivation: $E \Rightarrow -E \Rightarrow -(E) \Rightarrow -(E + E) \Rightarrow -(\text{id} + E) \Rightarrow -(\text{id} + \text{id})$



Ambiguity (二义性)

- If a grammar produces more than one parse tree for some sentence, it is ambiguous
- Example CFG: $E \rightarrow E + E \mid E * E \mid (E) \mid \text{id}$



Both are leftmost derivations

如果一个语法为某个句子生成了多个语法分析树，则它具有二义性

示例：该文法为 $\text{id}+\text{id}*\text{id}$ 生成了两种语法分析树。

Ambiguity (二义性) Cont.

- The grammar of a programming language typically needs to be unambiguous
 - Otherwise, there will be multiple ways to interpret a program
 - Given $E \rightarrow E + E \mid E * E \mid (E) \mid \text{id}$, how to interpret $a + b * c$?
- In some cases, it is convenient to use carefully chosen ambiguous grammars, together with disambiguating rules to discard undesirable parse trees
 - For example: multiplication before addition

程序设计语言的语法通常需要明确

- 否则，将有多种方法来解释程序
- 问题：给定 $E \rightarrow E + E \mid E * E \mid (E) \mid \text{id}$ ，如何解读 $a + b * c$?
- 答案：该文法是上一张 PPT 的示例，根据两种语法分析树，第一种解释：应先

处理 $b*c$ ，然后与 a 相加；第二种解释：应先处理 $a+b$ ，然后与 c 相乘

在某些情况下，使用精心选择的歧义语法以及消歧规则来丢弃不需要的语法分析树是很方便的

CFG vs. Regular Expressions

- **CFGs are more expressive than regular expressions**
 - Every construct that can be described by a regular expression can be described by a grammar, but not vice-versa
 - Every regular language is a context-free language, but not vice-versa
- Example: $L = \{a^n b^n \mid n > 0\}$
 - The language L can be described by CFG $S \rightarrow aSb \mid ab$
 - L cannot be described by regular expressions. In other words, we cannot construct a DFA to accept L

CFG 与正则表达式比较

CFG 比正则表达式更具表达能力

- 可以用正则表达式描述的每个构造都可以用语法描述，但反之不成立
- 每个正则语言都是上下文无关的语言，但反之不成立

示例: $L = \{a^n b^n \mid n > 0\}$

语言 L 可以用 CFG: $S \rightarrow aSb \mid ab$ 来描述

L 不能用正则表达式来描述。换句话说，我们不能构造一个 DFA 来接受 L

Proof by Contradiction

- Suppose there is a DFA D that accepts L and D has k states
- When processing a^{k+1} ..., D must enter a state s more than once (D enters one state after processing a symbol)¹
- Assume that D enters the state s after reading the i th and j th a ($i \neq j, i \leq k+1, j \leq k+1$)
- Since D accepts L , $a^i b^j$ must reach an accepting state. There must exist a path labeled b^+ from s to an accepting state
- Since a^i reaches the state s and there is a path labeled b^+ from s to an accepting state, D will accept $a^i b^j$. Contradiction!!!

¹ $a^{k+1}b^{k+1}$ is a string in L so D must accept it

反证法（了解即可）

假设存在 DFA D 接受 L , D 有 k 个状态

处理 $a^{k+1} \dots$ 时, D 必须进入状态 s , 且不止一次 (D 每处理一个符号都会进入一种状态)

假设 D 进入状态 s 在读入第 i 个 a 和第 j 个 a 后 ($i \neq j, i \leq k+1, j \leq k+1$)

按照上一张 PPT 的示例, 因为 D 接受文法 L , 所以 $a^i b^j$ 必须达到接受状态。必须存在一个标记为 b^j 从 s 到接受状态的路径

因为 a^i 到达了状态 s , 所以必有一条标记为 b^j 从 s 到接受状态的; 路径, D 将接受 $a^i b^j$ 。矛盾!!!

Any Regular Language Can Be Described by a CFG

- **(Proof by Construction)** Each regular language can be accepted by an NFA. We can construct a CFG to describe the language:
 - For each state i of the NFA, create a nonterminal symbol A_i
 - If state i has a transition to state j on input a , add the production $A_i \rightarrow aA_j$
 - If state i goes to state j on input ϵ , add the production $A_i \rightarrow A_j$
 - If i is an accepting state, add $A_i \rightarrow \epsilon$
 - If i is the start state, make A_i be the start symbol of the grammar

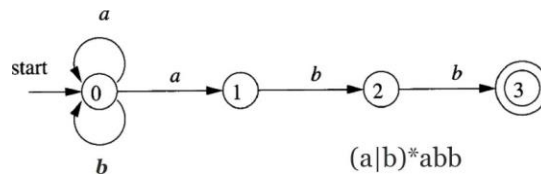
任何正则语言都可以用 CFG 描述

(结构证明) NFA 可以接受每种常规语言。我们可以构造一个 CFG 来描述语言:

- 对于每个在 NFA 中的状态 i , 创建一个非终结符 A_i
- 如果状态 i 在碰到输入 a 转为状态 j 时, 添加产生式 $A_i \rightarrow A_j$
- 如果状态 i 是一个接受状态, 添加产生式 $A_i \rightarrow \epsilon$
- 如果状态 i 是开始状态, 将 A_i 作为文法的开始符号

Example: NFA to CFG

- $A_0 \rightarrow aA_0 \mid bA_0 \mid aA_1$
- $A_1 \rightarrow bA_2$
- $A_2 \rightarrow bA_3$
- $A_3 \rightarrow \epsilon$



Consider the string **baabb**: The process of the NFA accepting the sentence corresponds exactly to the derivation of the sentence from the grammar

示例：NFA 到 CFG

考虑字符串 baabb: NFA 接受句子的过程与从语法中推导出句子的过程完全一致

Grammar Design

- CFGs are capable of describing most, but not all, of the syntax of programming languages
 - “Identifiers should be declared before use” cannot be described by a CFG
 - Subsequent phases must analyze the output of the parser to ensure compliance with such rules
- Before parsing, we typically apply several transformations to a grammar to make it more suitable for parsing
 - Eliminating ambiguity (消除二义性)
 - Eliminating left recursion (消除左递归)
 - Left factoring (提取左公因子)

CFG 能够描述编程语言的大部分（但不是全部）语法

- CFG 不能描述 “使用前应声明标识符”
- 后续阶段必须分析解析器的输出，以确保符合此类规则

在解析之前，我们通常对语法进行几种转换，使其更适合解析

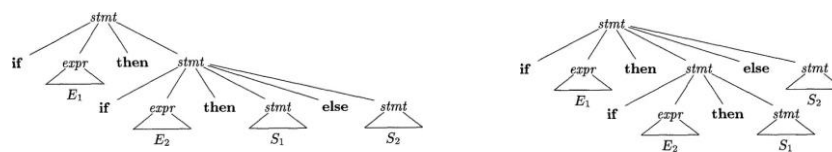
- 消除二义性
- 消除左递归
- 提取左公因子

Eliminating Ambiguity (1)

$stmt \rightarrow$

- `if expr then stmt`
- `if expr then stmt else stmt`
- `other`

Two parse trees for `if  $E_1$  then if  $E_2$  then  $S_1$  else  $S_2$`



Which parse tree is preferred in programming?
(i.e., `else` matches which `then`?)

消除二义性

问题：在编程中，哪个语法分析树是首选的？（即，`else` 匹配哪个 `then`？）

答案：第一种语法分析树

Eliminating Ambiguity (2)

- **Principle of proximity:** match each **else** with the closest unmatched **then**
 - **Idea of rewriting:** A statement appearing between a **then** and an **else** must be matched (must not end with an unmatched **then**)

```
stmt    →  matched_stmt  
         |  open_stmt  
matched_stmt →  if expr then matched_stmt else matched_stmt  
               |  other  
open_stmt  →  if expr then stmt  
               |  if expr then matched_stmt else open_stmt
```

Rewriting grammars to eliminate ambiguity is difficult.
There are no general rules to guide the process.



消除二义性

接近原则：用最接近的未匹配的 then 匹配 else

· 重写的想法：出现在 then 和 else 之间的语句必须匹配（不能以不匹配的 then 结束）

重写语法以消除歧义是困难的。没有指导这一过程的一般规则。

Eliminating Left Recursion

- A grammar is **left recursive** if it has a nonterminal A such that there is a derivation $A \Rightarrow^+ A\alpha$ for some string α
 - $S \rightarrow Aa \mid b$
 - $A \rightarrow Ac \mid Sd \mid \epsilon$
 - Because $S \Rightarrow Aa \Rightarrow Sda$
- **Immediate left recursion (立即左递归)**: the grammar has a production of the form $A \rightarrow A\alpha$
- Top-down parsing methods cannot handle left-recursive grammars (bottom-up parsing methods can handle...)

消除左递归

如果语法中存在一个非终结符 A ，使得某个字符 α 有相关的产生式 $A \Rightarrow^+ A\alpha$ ，则该语法为左递归语法

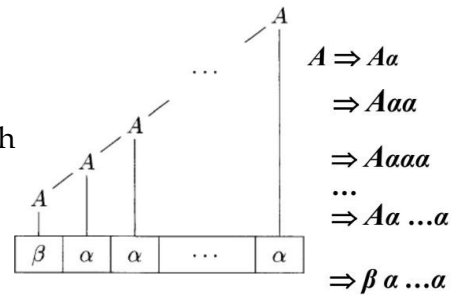
立即左递归: 语法有产生式 $A \rightarrow A\alpha$

自顶向下分析方法无法处理左递归语法（自底向上的分析方法可以处理...）

Eliminating Immediate Left Recursion

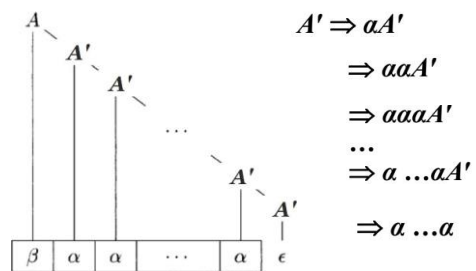
- Simple grammar: $A \rightarrow A\alpha \mid \beta$

- It generates sentences starting with the symbol β followed by zero or more α 's



- Replace the grammar by:

- $A \rightarrow \beta A'$
- $A' \rightarrow \alpha A' \mid \epsilon$
- It is right recursive now



消除立即左递归

简单语法: $A \rightarrow A\alpha \mid \beta$ 它生成以符号 β 开头的句子, 后面跟零个或更多的 α 's

将语法替换为:

$A \rightarrow \beta A'$

$A' \rightarrow \alpha A' \mid \epsilon$

它现在是右递归了

Eliminating Immediate Left Recursion

- The general case: $A \rightarrow A\alpha_1 \mid \dots \mid A\alpha_m \mid \beta_1 \mid \dots \mid \beta_n$
- Replace the grammar by:
 - $A \rightarrow \beta_1 A' \mid \dots \mid \beta_n A'$
 - $A' \rightarrow \alpha_1 A' \mid \dots \mid \alpha_m A' \mid \epsilon$

消除立即左递归

$$\bullet A \rightarrow A\alpha_1 \mid \dots \mid A\alpha_m \mid \beta_1 \mid \dots \mid \beta_n$$

将语法替换为:

$$A \rightarrow \beta_1 A' \mid \dots \mid \beta_n A'$$

$$A' \rightarrow \alpha_1 A' \mid \dots \mid \alpha_m A' \mid \epsilon$$

Example

$$\begin{array}{lcl}
 E \rightarrow E + T \mid T & & E \rightarrow T E' \\
 \quad \quad \quad \underbrace{\quad}_\alpha \quad \underbrace{\quad}_\beta & & E' \rightarrow + T E' \mid \epsilon \\
 T \rightarrow T * F \mid F & \longrightarrow & T \rightarrow F T' \\
 \quad \quad \quad \underbrace{\quad}_\alpha \quad \underbrace{\quad}_\beta & & T' \rightarrow * F T' \mid \epsilon \\
 F \rightarrow (E) \mid \text{id} & & F \rightarrow (E) \mid \text{id}
 \end{array}$$

$$\begin{array}{lcl}
 \boxed{E \rightarrow E + T \mid T} & \Longrightarrow & \boxed{E \rightarrow T E'} \\
 \boxed{T \rightarrow T * F \mid F} & & \boxed{E' \rightarrow + T E' \mid \epsilon} \\
 F \rightarrow (E) \mid \text{id} & & \boxed{T \rightarrow F T'} \\
 & & \boxed{T' \rightarrow * F T' \mid \epsilon} \\
 & & F \rightarrow (E) \mid \text{id}
 \end{array}$$

Eliminating Left Recursion

- The technique for eliminating immediate left recursion does not work for the non-immediate left recursions
- The general left recursion eliminating algorithm (**iterative**)
 - Input:** Grammar G with no cycles or ϵ -productions
 - Output:** An equivalent grammar with no left recursion

arrange the nonterminals in some order $A_1, A_2, \dots, A_n$.

for (each i from 1 to n) {
 for (each j from 1 to $i - 1$) {
 replace each production of the form $A_i \rightarrow A_j \gamma$ by the
 productions $A_i \rightarrow \delta_1 \gamma \mid \delta_2 \gamma \mid \dots \mid \delta_k \gamma$, where
 $A_j \rightarrow \delta_1 \mid \delta_2 \mid \dots \mid \delta_k$ are all current A_j -productions
 }
 eliminate the immediate left recursion among the A_i -productions
 }

消除立即左递归的技术不适用于非立即左递归

虽然形式上产生式没有直接左递归，但由于形成了环，导致进行推导后后出现左递归。

通用左递归消除算法（迭代）

- 输入：没有循环或 ϵ -产生式的语法 G
- 输出：没有左递归的等价语法

以下是算法

对于每个非终结符 A_i ，检查是否有形如 $A_i \rightarrow A_j v$ 的产生式（ A_j 是其他的非终结符）

将 $A_i \rightarrow A_j v$ 修改为 $A_i \rightarrow \delta_1 v \mid \delta_2 v \mid \cdots \mid \delta_k v$

如果 $A_j \rightarrow \delta_1 \mid \delta_2 \mid \cdots$

接着消除 A_i 的所有直接左递归

Example

$S \rightarrow Aa \mid b \quad A \rightarrow Ac \mid Sd \mid \epsilon$

- Order the nonterminals: S, A
- $i = 1$:
 - The inner loop does not run; there is no immediate left recursion among S -productions
- $i = 2$:
 - $j = 1$, replace the production $A \rightarrow Sd$ by $A \rightarrow Aad \mid bd$
 - $A \rightarrow Aad \mid bd \mid Ac \mid \epsilon$
 - Eliminate immediate left recursion

$$\begin{aligned} S &\rightarrow Aa \mid b \\ A &\rightarrow bdA' \mid A' \\ A' &\rightarrow cA' \mid adA' \mid \epsilon \end{aligned}$$

The example grammar contains an ϵ -production, but it is harmless

Left Factoring (提取左公因子)

- If we have the following two productions

$$\begin{aligned} stmt &\rightarrow \text{if } expr \text{ then } stmt \text{ else } stmt \\ &| \text{ if } expr \text{ then } stmt \end{aligned}$$

- On seeing input **if**, we cannot immediately decide which production to choose
- In general, if $A \rightarrow \alpha\beta_1 \mid \alpha\beta_2$ are two productions, and the input begins with a nonempty string derived from α . We may defer choosing productions by expanding A to $\alpha A'$ first

$\begin{aligned} A &\rightarrow \alpha A' \\ A' &\rightarrow \beta_1 \mid \beta_2 \end{aligned}$
--

如果我们有以下两个产生式

看到输入时**if**，我们无法立即决定选择哪个产生式

一般来说，如果 $A \rightarrow \alpha\beta_1 \mid \alpha\beta_2$ 是两个产生式，并且输入以从 α 派生的非空字符串开始。

我们可以通过扩大 A 到 $\alpha A'$ 来推迟选择产生式

通过改写产生式来推迟决策，直到输入足够多的信息后再做出。

Algorithm: Left Factoring a Grammar

- **Input:** Grammar G
- **Output:** An equivalent left-factored grammar
- For each nonterminal A , find the longest prefix α common to two or more of its alternatives.
- If $\alpha \neq \epsilon$, replace all A -productions $A \rightarrow \alpha\beta_1 \mid \alpha\beta_2 \mid \dots \mid \alpha\beta_n \mid \gamma$, where γ represents all alternatives that do not begin with α , by

$$\begin{aligned} A &\rightarrow \alpha A' \mid \gamma \\ A' &\rightarrow \beta_1 \mid \beta_2 \mid \dots \mid \beta_n \end{aligned}$$

- Repeatedly apply the above transformation until no two alternatives for a nonterminal have a common prefix

算法：语法的左公因子提取

输入：语法 G

输出：等效的左公因子提取语法

- 对于每个非终结符 A ，查找最长的前缀 α ，它是两个或多个替换项的公共前缀。
- 重复转换，直到非终结符的两个替代项都没有公共前缀为止

Top-Down Parsing

- **Problem definition:** Constructing a parse tree for the input string, starting from the root and creating the nodes of the parse tree in preorder (depth-first)
 - Equivalent to **finding a leftmost derivation for an input string**
- **Basic idea (two actions):**
 - **Predict:** At each step of parsing, determine the production to be applied for the leftmost nonterminal
 - **Match:** Match the terminals in the chosen production's body with the input string

自顶向下语法分析

问题定义：为输入字符串构建一个语法分析树，从根开始，按预序创建语法分析树的节点（深度优先）

- 相当于为输入字符串找到最左推导

基本思想（两个步骤）：

- 预测：在解析的每一步，确定最左边非终结符所用的产生式
- 匹配：将所选产生式主体中的终结符与输入字符串相匹配

Top-Down Parsing Example

- Grammar**

$$E \rightarrow TE'$$

$$E' \rightarrow +TE' \mid \epsilon$$

$$T \rightarrow FT'$$

$$T' \rightarrow *FT' \mid \epsilon$$

$$F \rightarrow (E) \mid id$$

- Input string**

id + id * id

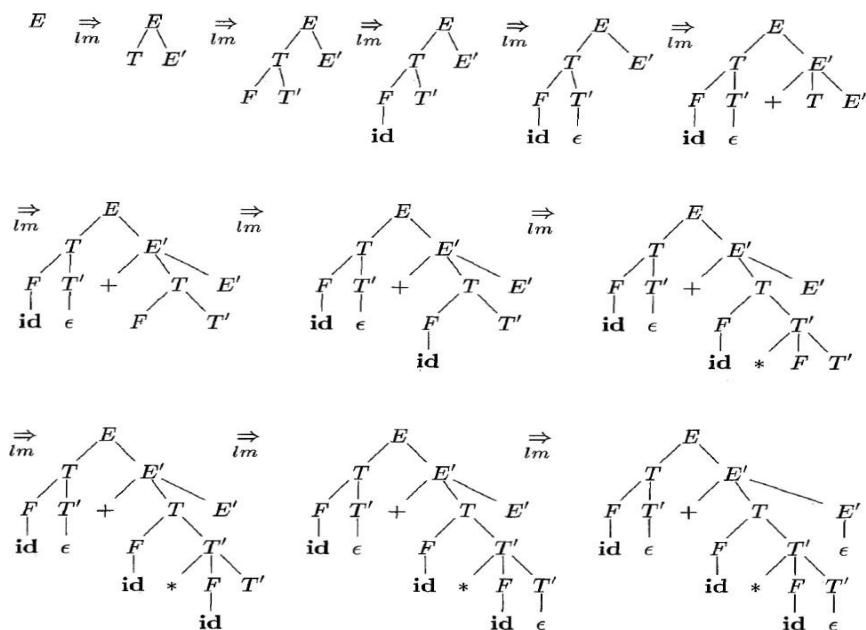
Is the input string a sentence of the grammar?



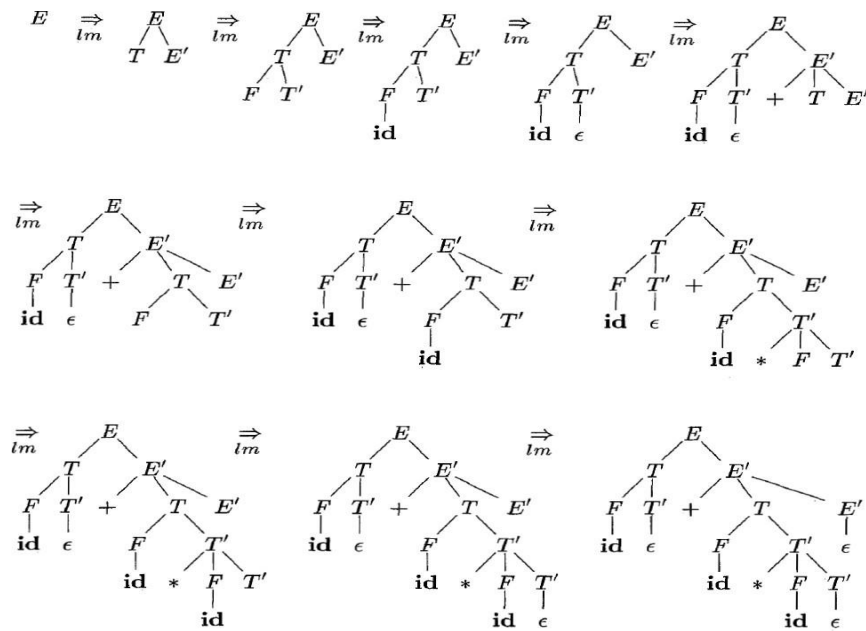
问题：输入串是该文法的句子吗？

答案：是，语法分析树见下一页 PPT

- Grammar:** $E \rightarrow TE'$ $E' \rightarrow +TE' \mid \epsilon$ $T \rightarrow FT'$ $T' \rightarrow *FT' \mid \epsilon$ $F \rightarrow (E) \mid id$
Input string: id + id * id



Key decision in top-down parsing: Which production to apply at each step?



Fall 2024

BUPT Compilers

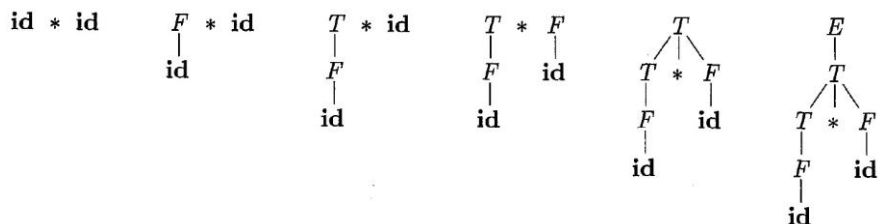
42

自顶向下解析中的关键决策：在每一步应用哪个产生式？

Bottom-Up Parsing

$$\begin{aligned}
 E &\rightarrow E + T \mid T \\
 T &\rightarrow T * F \mid F \\
 F &\rightarrow (E) \mid \text{id}
 \end{aligned}$$

- **Problem definition:** Constructing a parse tree for an input string beginning at the leaves (**terminals**) and working up towards the root (**start symbol of the grammar**)
 - Equivalent to **finding a rightmost derivation (in reverse) for an input string**
- **Shift-reduce parsing (移入-归约分析技术)** is a **general style** of bottom-up parsing (using a **stack** to hold grammar symbols)



Fall 2024

BUPT Compilers

43

自底向上语法分析

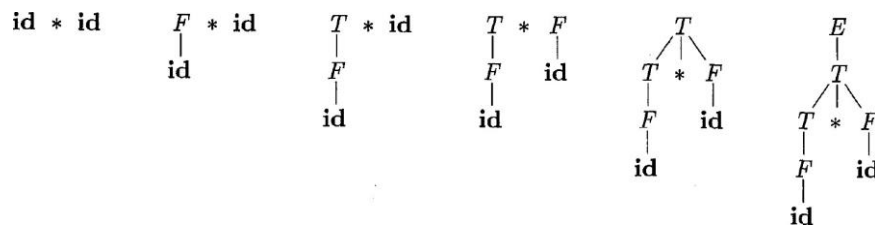
问题定义：为输入字符串构建一个从叶子（终结符）开始并向上延伸到根（语法的开始符号）的语法分析树

- 相当于为输入字符串找到最右推导（相反）

移入-归约分析技术是自底向上语法分析的通用方式（使用栈来保存语法符号）

Reductions (归约)

- Bottom-up parsing can be seen as a process of “reducing” a string ω to the start symbol of the grammar
- At each *reduction step*, a specific substring (at the top of the stack) matching the body of a production is replaced by the head of the production (the reverse of a step in derivation)



自底向上语法分析可以看作是一个将字符串 ω “归约”到开始符号的过程

在每一个归约步骤中，与产生式主体匹配的特定子串（位于堆栈顶部）都会被产生式的头部替换（与推导步骤相反）

Shift-Reduce Parsing Example

Parsing steps on input $\text{id}_1 * \text{id}_2$

STACK	INPUT	ACTION
\$	$\text{id}_1 * \text{id}_2$ \$	shift
\$ id_1	$* \text{id}_2$ \$	reduce by $F \rightarrow \text{id}$
\$ F	$* \text{id}_2$ \$	reduce by $T \rightarrow F$
\$ T	$* \text{id}_2$ \$	shift
\$ $T *$	id_2 \$	shift
\$ $T * \text{id}_2$	\$	reduce by $F \rightarrow \text{id}$
\$ $T * F$	\$	reduce by $T \rightarrow T * F$
\$ T	\$	reduce by $E \rightarrow T$
\$ E	\$	accept

$E \rightarrow E + T \mid T$
 $T \rightarrow T * F \mid F$
 $F \rightarrow (E) \mid \text{id}$

Key decisions:

1. When to reduce?
2. Which production to apply?

移入-归约分析技术示例

关键决策：

1. 什么时候归约？
2. 应用哪个产生式？

Recursive-Descent Parsing

(递归下降的语法分析)

- A recursive-descent parsing program has a set of procedures, one for each nonterminal
 - The procedure for a nonterminal scans the structure (a substring of the input) corresponding to the nonterminal
- Execution begins with the procedure for the start symbol
 - Announce success if the procedure scans the entire input string

```
void A() { // a typical procedure for a nonterminal
1)      Choose an A-production,  $A \rightarrow X_1X_2 \cdots X_k$ ;
2)      for (  $i = 1$  to  $k$  ) {
3)          if (  $X_i$  is a nonterminal )
4)              call procedure  $X_i()$ ;
5)          else if (  $X_i$  equals the current input symbol  $a$  )
6)              advance the input to the next symbol;
7)          else /* an error has occurred */;
      }
}
```

Fall 2024

BUPT Compilers

47

递归下降的语法分析程序对于每个非终结符都有一套程序

- 该程序会扫描与非终结符对应的结构（输入的子字符串）

从开始符号的程序开始执行

- 如果程序扫描整个输入字符串，则宣布成功

Backtracking (回溯)

```
void A() {  
1)    Choose an A-production,  $A \rightarrow X_1X_2 \cdots X_k$ ;  
2)    for (  $i = 1$  to  $k$  ) {  
3)        if (  $X_i$  is a nonterminal )  
4)            call procedure  $X_i()$ ;  
5)        else if (  $X_i$  equals the current input symbol  $a$  )  
6)            advance the input to the next symbol;  
7)    else /* an error has occurred */;  
    }  
}
```



If there is a failure at line 7, does this mean that there must be syntax errors?

问题：如果第 7 行出现故障，这是否意味着一定存在语法错误？

答案：不一定。见下一页 PPT

Backtracking (回溯)

```
void A() {  
1)    Choose an A-production,  $A \rightarrow X_1X_2 \cdots X_k$ ;  
2)    for (  $i = 1$  to  $k$  ) {  
3)        if (  $X_i$  is a nonterminal )  
4)            call procedure  $X_i()$ ;  
5)        else if (  $X_i$  equals the current input symbol  $a$  )  
6)            advance the input to the next symbol;  
7)    else /* an error has occurred */;  
    }  
}
```



The failure might be caused by a wrong choice of A-production at line 1 !!!

故障可能是由于在底=第 1 行选错了 A-产生式

Backtracking (回溯)

- General recursive-descent parsing may require **repeated scans** over the input (**backtracking**)
- To allow backtracking, we need to modify the algorithm

```
void A() {  
1) Choose an A-production,  $A \rightarrow X_1X_2 \cdots X_k$ ;  
2)   for (  $i = 1$  to  $k$  ) {  
3)       if (  $X_i$  is a nonterminal )  
4)           call procedure  $X_i()$ ;  
5)       else if (  $X_i$  equals the current input symbol  $a$  )  
6)           advance the input to the next symbol;  
7)       else /* an error has occurred */;  
   }  
}
```

Instead of exploring one A-production, we must try each possible production in some order.

一般的递归下降语法分析可能需要对输入进行重复扫描，以尝试不同的产生式（回溯）

为了允许回溯，我们需要修改算法

与考察 A-产生式不同，我们必须按照一定的顺序尝试每一种可能的产生式。

Backtracking (回溯)

- General recursive-descent parsing may require **repeated scans** over the input (**backtracking**)
- To allow backtracking, we need to modify the algorithm

```
void A() {  
1)    Choose an A-production,  $A \rightarrow X_1X_2 \cdots X_k$ ;  
2)    for (  $i = 1$  to  $k$  ) {  
3)        if (  $X_i$  is a nonterminal )  
4)            call procedure  $X_i()$ ;  
5)        else if (  $X_i$  equals the current input symbol  $a$  )  
6)            advance the input to the next symbol;  
7)        else /* an error has occurred */;  
    }  
}
```

When there is a failure at line 7, return to line 1 and try another A-production.

当第 7 行出现故障时，返回第 1 行并尝试另一个 A 产生式

Backtracking (回溯)

- General recursive-descent parsing may require **repeated scans** over the input (**backtracking**)
- To allow backtracking, we need to modify the algorithm

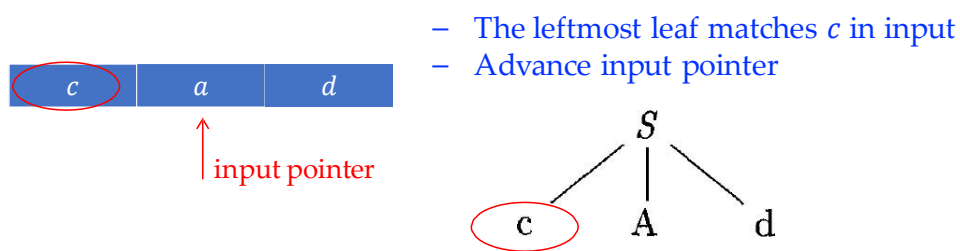
```
void A() {  
1)    Choose an A-production,  $A \rightarrow X_1X_2 \cdots X_k$ ;  
2)    for (  $i = 1$  to  $k$  ) {  
3)        if (  $X_i$  is a nonterminal )  
4)            call procedure  $X_i()$ ;  
5)        else if (  $X_i$  equals the current input symbol  $a$  )  
6)            advance the input to the next symbol;  
7)        else /* an error has occurred */;  
    }  
}
```

In order to try another A-production, we must reset the input pointer that points to the next symbol to scan (**failed trials consume symbols**)

为了尝试另一个 A-产生式时，我们必须重置指向下一个要扫描的符号的输入指针（失败尝试的消耗符号）

Backtracking Example

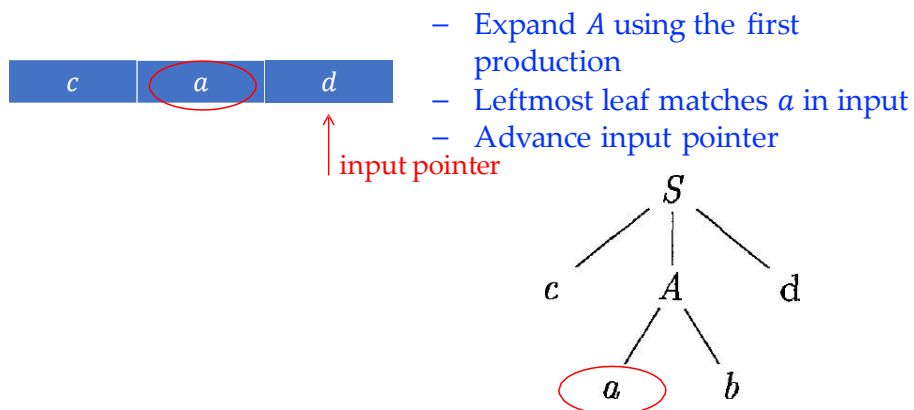
- Grammar: $S \rightarrow cAd$ $A \rightarrow ab \mid a$
- Input string: cad



S 结点最左边的叶子匹配输入中的 c
移进输入指针

Backtracking Example

- Grammar: $S \rightarrow cAd$ $A \rightarrow ab \mid a$
- Input string: cad



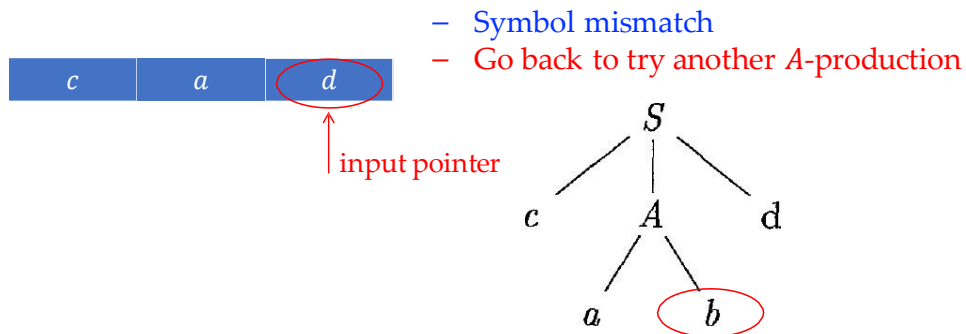
使用第 1 个产生式扩展 A

A 结点最左边的叶子匹配输入中的 a

移进输入指针

Backtracking Example

- Grammar: $S \rightarrow cAd$ $A \rightarrow ab \mid a$
- Input string: cad



Fall 2024

BUPT Compilers

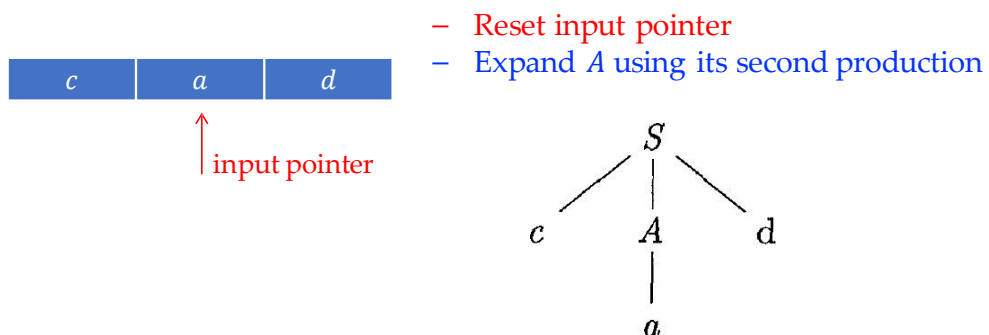
55

指针指向的 d 与符号 b 不匹配

返回尝试另一个 A -产生式

Backtracking Example

- Grammar: $S \rightarrow cAd$ $A \rightarrow ab \mid a$
- Input string: cad



Fall 2024

BUPT Compilers

56

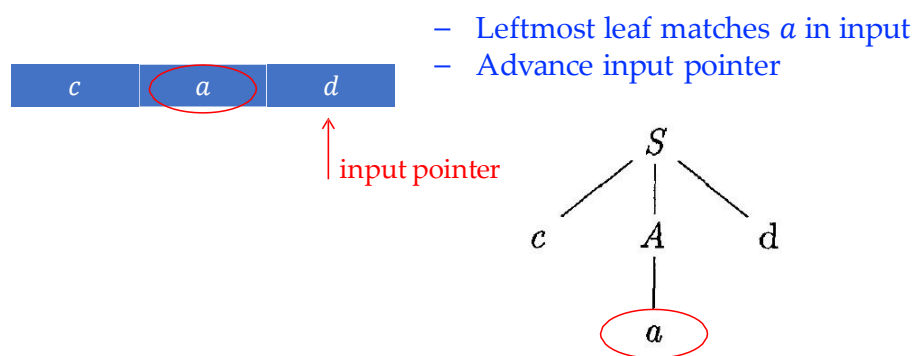
重置输入指针的位置

使用 A 的第二个产生式扩展 A

Backtracking Example

- **Grammar:** $S \rightarrow cAd$ $A \rightarrow ab \mid a$

- **Input string:** cad



A 结点最左边的叶子匹配输入中的 a

移进输入指针

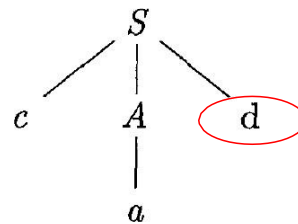
Backtracking Example

- Grammar: $S \rightarrow cAd$ $A \rightarrow ab \mid a$
- Input string: cad



- The last leaf node matches d in input
- Announce success!

Scanned entire input



最后的叶子匹配输入中的 d

成功

The Problem of Left Recursion

Suppose there is $A \rightarrow A\alpha \dots$

```
void A() {  
1)    Choose an A-production,  $A \rightarrow X_1X_2 \dots X_k$ ;  
2)    for (  $i = 1$  to  $k$  ) {  
3)        if (  $X_i$  is a nonterminal )  
4)            call procedure  $X_i()$ ;  
5)        else if (  $X_i$  equals the current input symbol  $a$  )  
6)            advance the input to the next symbol;  
7)        else /* an error has occurred */;  
    }  
}
```

If there is **left recursion** in a CFG, a recursive-descent parser may go into **an infinite loop**! Revise the CFG before parsing!!!

左递归带来的问题

如果 CFG 中有左递归，递归下降的语法分析器可能会进入无限循环，所以需要解析前修改 CFG

Can We Avoid Backtracking?

```
void A() {  
1)   Choose an  $A$ -production,  $A \rightarrow X_1X_2 \cdots X_k$ ;  
2)   for (  $i = 1$  to  $k$  ) {  
3)       if (  $X_i$  is a nonterminal )  
4)           call procedure  $X_i()$ ;  
5)       else if (  $X_i$  equals the current input symbol  $a$  )  
6)           advance the input to the next symbol;  
7)       else /* an error has occurred */;  
    }  
}
```

Key problem: At line 1, we make *random choices* (brute force search)

我们能避免回溯吗?

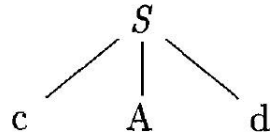
关键问题：在第 1 行，我们进行随机选择（暴力搜索）

Can We Avoid Backtracking?

- Grammar: $S \rightarrow cAd$ $A \rightarrow c \mid a$
- Input string: cad



↑
input pointer



When rewriting A , is it a good idea to choose $A \rightarrow c$?

No! If we look ahead, the next char in the input is a .

$A \rightarrow c$ is obviously a bad choice!!!

重写时 A , 选择产生式 $A \rightarrow c$ 好吗?

不, 如果我们向前看, 输入中的下一个字符是 a

$A \rightarrow c$ 显然是个糟糕的选择

Looking Ahead Helps!

- Suppose the input string is $x\mathbf{a}$
- Suppose the current sentential form is $x\mathbf{A}\beta$
 - $\mathbf{A}$ is a non-terminal; β may contain both terminals and non-terminals

If we know the following fact for the productions $\mathbf{A} \rightarrow \alpha \mid \gamma$:

- $a \in FIRST(\alpha)$: α derives strings that begin with a
- $a \notin FIRST(\gamma)$: γ derives strings that do not begin with a

**FIRST(α) denotes the set of terminals that begin strings derived from α*

After matching x , we should choose $\mathbf{A} \rightarrow \alpha$ to rewrite $\mathbf{A}$

向前看有用

假设输入字符串为 $x\mathbf{a}$

假设当前的句型是 $x\mathbf{A}\beta$

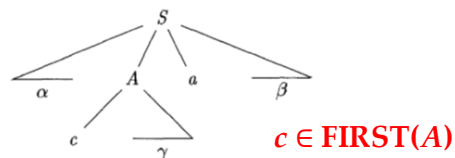
- $\mathbf{A}$ 是非终结符； β 可以同时包含终结符和非终结符

如果我们知道产生式 $\mathbf{A} \rightarrow \alpha \mid \gamma$ 的以下事实:

- $a \in FIRST(\alpha)$: α 可以推导以 a 开头的字符串
- $a \notin FIRST(\gamma)$: γ 不可以推导出以 a 开头的字符串

**FIRST(α)* 表示可从 α 推导得到的串的首符号的集合

FIRST



- **FIRST(α)**, where α is any string of grammar symbols

- The set of **terminals** that **begin strings derived from α**
- If $\alpha \Rightarrow^* \epsilon$, then ϵ is also in $\text{FIRST}(\alpha)$

▪ FIRST function is useful in parsing

- $A \rightarrow \alpha \mid \beta$; suppose $\text{FIRST}(\alpha)$ and $\text{FIRST}(\beta)$ are disjoint
- Choose $A \rightarrow \alpha$ if the next input symbol $a \in \text{FIRST}(\alpha)$
- Choose $A \rightarrow \beta$ if the next input symbol $a \in \text{FIRST}(\beta)$

$\text{FIRST}(\alpha)$, α 是语法符号串

如果 α 是终结符, FIRST 集为字符串首字母

如果 α 可以推导为空, 则空在 FIRST 集中

FIRST 集在语法分析是有用的

$A \rightarrow \alpha \mid \beta$, 假设 $\text{FIRST}(\alpha)$ and $\text{FIRST}(\beta)$ 交集为空

如果下一个 a 在 $\text{FIRST}(\alpha)$ 中, 选择产生式 $A \rightarrow \alpha$

如果下一个 a 在 $\text{FIRST}(\beta)$ 中, 选择产生式 $A \rightarrow \beta$

Computing FIRST

- **FIRST(X)**, where X is a grammar symbol
 - If X is a **terminal**, then $\text{FIRST}(X) = \{X\}$
 - If X is a **nonterminal** and $X \rightarrow Y_1 Y_2 \dots Y_k$ ($k \geq 1$) is a production
 - If for some i , a is in $\text{FIRST}(Y_i)$ and ϵ is in all of $\text{FIRST}(Y_1), \dots, \text{FIRST}(Y_{i-1})$, then add a to $\text{FIRST}(X)$
 - If ϵ is in all of $\text{FIRST}(Y_1), \dots, \text{FIRST}(Y_k)$, then add ϵ to $\text{FIRST}(X)$
 - If X is a **nonterminal** and $X \rightarrow \epsilon$, then add ϵ to $\text{FIRST}(X)$

计算 FIRST 集

计算 **FIRST(X)** X 是语法符号

- 如果 X 是一个终结符，那么 $\text{FIRST}(X) = \{X\}$
- 如果 X 是一个非终结符，且 $X \rightarrow Y_1 Y_2 \dots Y_k$ ($k \geq 1$) 是一个产生式
 - 如果对于某个 i ， a 在 $\text{FIRST}(Y_i)$ 中且 ϵ 在所有的 $\text{FIRST}(Y_1), \dots, \text{FIRST}(Y_{i-1})$

中，就把 a 加入到 $\text{FIRST}(X)$ 中

- 如果对于所有的 $\text{FIRST}(Y_1), \dots, \text{FIRST}(Y_k)$ ， ϵ 在其中，那么将 ϵ 加入到 $\text{FIRST}(X)$ 中

- 如果 X 是一个非终结符，并且存在产生式 $X \rightarrow \epsilon$ ，那么将 ϵ 加入到 $\text{FIRST}(X)$ 中

Computing FIRST Cont.

- **FIRST($X_1X_2 \dots X_n$)**, where $X_1X_2 \dots X_n$ is a string of grammar symbols
 - Add all **non- ϵ symbols** of FIRST(X_1) to FIRST($X_1X_2 \dots X_n$)
 - If **ϵ is in FIRST(X_1)**, add non- ϵ symbols of FIRST(X_2) to FIRST($X_1X_2 \dots X_n$); If **ϵ is in FIRST(X_1) and FIRST(X_2)**, add non- ϵ symbols of FIRST(X_3) to FIRST($X_1X_2 \dots X_n$); ...
 - If **ϵ is in FIRST(X_i) for all i** , add ϵ to FIRST($X_1X_2 \dots X_n$)

计算 FIRST 集

计算 **FIRST($X_1X_2 \dots X_n$)** $X_1X_2 \dots X_n$ 是语法符号的串

- 向 FIRST($X_1X_2 \dots X_n$)加入 FIRST(X_1)中所有的非 ϵ 符号
- 如果 ϵ 在 FIRST(X_1)中，再加入 FIRST(X_2)中所有的非 ϵ 符号；如果 ϵ 在 FIRST(X_1)和 FIRST(X_2)中，加入 FIRST(X_3)中的所有非 ϵ 符号，以此类推。
- 最后，如果对于所有的 i ， ϵ 都在 FIRST(X_i)中，那么将加入到 FIRST($X_1X_2 \dots X_n$)中。

FIRST Example

- Grammar

- $E \rightarrow TE'$ $E' \rightarrow +TE' \mid \epsilon$
- $T \rightarrow FT'$ $T' \rightarrow *FT' \mid \epsilon$ $F \rightarrow (E) \mid \text{id}$

- FIRST sets

- $\text{FIRST}(F) = \{ (, \text{id} \}$
- $\text{FIRST}(T) = \text{FIRST}(F) = \{ (, \text{id} \}$
- $\text{FIRST}(E) = \text{FIRST}(T) = \{ (, \text{id} \}$
- $\text{FIRST}(E') = \{ +, \epsilon \}$ $\text{FIRST}(T') = \{ *, \epsilon \}$
- $\text{FIRST}(TE') = \text{FIRST}(T) = \{ (, \text{id} \}$
- ...

Looking Ahead Helps Cont.

- Suppose the input string is $x\mathbf{a}$
- Suppose the current sentential form is $x\mathbf{A}\beta$
 - $\mathbf{A}$ is a non-terminal; β may contain both terminals and non-terminals

If we know that for the production $\mathbf{A} \rightarrow \alpha$, $\epsilon \in \text{FIRST}(\alpha)$, can we choose the production to rewrite $\mathbf{A}$?

Yes, only if β can derive strings beginning with a , that is, $\mathbf{A}$ can be followed by a in some sentential forms (i.e., $\mathbf{a} \in \text{FOLLOW}(\mathbf{A})$)

假设输入字符串为 xa

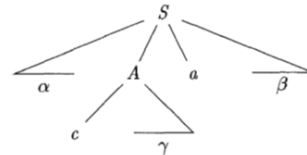
假设当前的句型是 $xA\beta$

- A 是非终结符； β 可以同时包含终结符和非终结符

如果我们知道生产 $A \rightarrow \alpha, \epsilon \in FIRST(\alpha)$, 我们能选择重写 A 的产生式吗?

可以, 只有在 β 可以推导以 a 开头的字符串, 即, 在一些句型中 A 后面可以跟着 a
(即, $a \in FOLLOW(A)$)

FOLLOW



- **FOLLOW(A)**, where A is a nonterminal $a \in FOLLOW(A)$

- The set of **terminals** a that can appear immediately to the right of A in some sentential form

- **FOLLOW function is also useful in parsing**

- Consider a production $A \rightarrow \alpha$; when $\alpha \xRightarrow{*} \epsilon$, FOLLOW(A) can help choose the right production
- **Example**: if the next input symbol is b , and $b \in FOLLOW(A)$, then we can choose $A \rightarrow \alpha$

FOLLOW(A), A 是非终结符

$a \in FOLLOW(A)$ 表示终结符在某些句型中可以出现在 A 的右边

对产生式 $A \rightarrow \alpha$, 当 $\alpha \Rightarrow^* \epsilon$, FOLLOW(A)可以帮助我们选择正确的产生式

Computing FOLLOW

- Computing FOLLOW(A) for all nonterminals A

- Add $\$$ in FOLLOW(S), where S is the start symbol and $\$$ is the input right endmarker
- Apply the rules below, until all FOLLOW sets do not change
 1. If there is a production $A \rightarrow \alpha B \beta$, then everything in FIRST(β) except ϵ is in FOLLOW(B)
 2. If there is a production $A \rightarrow \alpha B$ (or $A \rightarrow \alpha B \beta$ and FIRST(β) contains ϵ) then everything in FOLLOW(A) is in FOLLOW(B)

ϵ will not appear in any FOLLOW set

计算 FOLLOW 集

计算所有非终结符 A 的 FOLLOW(A)集合

- 将 $\$$ 添加到 FOLLOW (S) 中，其中 S 是开始符号， $\$$ 是输入右端的结束标记
- 不断应用下面的规则，直到所有 FOLLOW 集不再改变
- 如果存在一个产生式 $A \rightarrow \alpha B \beta$ ，那么 FIRST(β)中除 ϵ 之外的所有符号都在

FOLLOW(B)中

- 如果存在一个产生式 $A \rightarrow \alpha B$ ，或存在产生式 $A \rightarrow \alpha B \beta$ 且 FIRST(β)包含 ϵ ，

那么 FOLLOW(A)中的所有符号都在 FOLLOW(B)中

ϵ 不会在任何 FOLLOW 集中

FOLLOW Example

- Grammar

- $E \rightarrow TE'$ $E' \rightarrow +TE' \mid \epsilon$
- $T \rightarrow FT'$ $T' \rightarrow *FT' \mid \epsilon$ $F \rightarrow (E) \mid \text{id}$

- FOLLOW sets

- FOLLOW(E) = { $\$,)$ }
- FOLLOW(E') = { $\$,)$ }
- FOLLOW(T) = { $+, \$,)$ }
- FOLLOW(T') = { $+, \$,)$ }
- FOLLOW(F) = { $*, +, \$,)$ }

- $\$$ is always in FOLLOW(E)
- Everything in FIRST($)$ except ϵ is in FOLLOW(E)

- Everything in FIRST(E') except ϵ is in FOLLOW(T)
- Since $E' \rightarrow \epsilon$, everything in FOLLOW(E) is in FOLLOW(T)

LL(1) Grammars

- Recursive-descent parsers needing no backtracking can be constructed for a class of grammars called **LL(1)**
 - **1st L**: scanning the input from left to right
 - **2nd L**: producing a leftmost derivation
 - **1**: using one input symbol of lookahead at each step to make parsing decision

我们可以为一类称为 LL(1)的语法构造不需要回溯的递归下降语法分析器

从左到右扫描输入

产生最左推导

在每一步使用一个前瞻输入符号来作出分析决策

LL(1) Grammars Cont.

A grammar G is LL(1) if and only if for any two distinct productions $A \rightarrow \alpha \mid \beta$, the following conditions hold:

1. There is no terminal a such that α and β derive strings beginning with a
2. At most one of α and β can derive the empty string
3. If $\beta \Rightarrow^* \epsilon$, then α does not derive any string beginning with a terminal in $\text{FOLLOW}(A)$ and vice versa

* The three conditions basically rule out the possibility of applying both productions

More formally:

1. $\text{FIRST}(\alpha) \cap \text{FIRST}(\beta) = \emptyset$ (conditions 1-2 above)
2. If $\epsilon \in \text{FIRST}(\beta)$, then $\text{FIRST}(\alpha) \cap \text{FOLLOW}(A) = \emptyset$ and vice versa

语法 G 是 LL(1) 当且仅当对于任意两个不同的产生式 $A \rightarrow \alpha \mid \beta$, 以下条件成立:

1. 不存在终结符 a 使得 α 和 β 都可以推导以 a 开头的字符串
2. α 和 β 中最多只有一个可以推导空字符串
3. 如果 $\beta \Rightarrow^* \epsilon$, 那么 α 不能推导出任何以 $\text{FOLLOW}(A)$ 中某个终结符开头的字符串, 反之亦然

*这三个条件基本上排除了应用两个产生式的可能性

更正式地说:

1. $\text{FIRST}(\alpha) \cap \text{FIRST}(\beta) = \emptyset$ (上述条件 1-2)

如果 $\epsilon \in \text{FIRST}(\beta)$, 那么 $\text{FIRST}(\alpha) \cap \text{FOLLOW}(A) = \emptyset$, 反之亦然

LL(1) Grammars Cont.

- For LL(1) grammars, during recursive-descent parsing, the proper production to apply for a nonterminal can be selected by looking only at the current input symbol

Grammar: $stmt \rightarrow \text{if}(\text{expr})\ stmt\ \text{else}\ stmt \mid \text{while}(\text{expr})\ stmt \mid \text{a}$

①

②

③

Parsing steps for input: `if(expr) while(expr) a else a`

1. Rewrite the start symbol *stmt* with ①: **if(expr) stmt else stmt**
2. Rewrite the leftmost *stmt* with ②: **if(expr) while(expr) stmt else stmt**
3. Rewrite the leftmost *stmt* with ③: **if(expr) while(expr) a else stmt**
4. Rewrite the leftmost *stmt* with ③: **if(expr) while(expr) a else a**

对于 LL(1)语法，在递归下降语法分析过程中，只需查看当前输入符号，就可以选择应用于非终结符的正确产生式：

给定文法

给出输入的分析步骤:

- 1.用产生式①重写开始符号 $stmt$: **if(expr)stmt else stmt**
- 2.用产生式②重写最左边的 $stmt$: **if(expr)while(expr) stmt else stmt**
- 3.用产生式③重写最左边的 $stmt$: **if(expr)while(expr) a else stmt**
- 4.用产生式③重写最左边的 $stmt$: **if(expr)while(expr) a else a**

Parsing Table (预测分析表)

- Recursive-descent parsers (or **LL parsers**) are table-based parsers
- A predictive **parsing table** is a two-dimensional array that determines which production the parser should choose when it sees a nonterminal A and a symbol a on its input stream
- The parsing table of an LL(1) parser has **no entries with multiple productions**

NON - TERMINAL	INPUT SYMBOL					
	id	+	*	(	)	\$
E	$E \rightarrow TE'$			$E \rightarrow TE'$		
E'		$E' \rightarrow +TE'$			$E' \rightarrow \epsilon$	$E' \rightarrow \epsilon$
T	$T \rightarrow FT'$			$T \rightarrow FT'$		
T'		$T' \rightarrow \epsilon$	$T' \rightarrow *FT'$		$T' \rightarrow \epsilon$	$T' \rightarrow \epsilon$
F	$F \rightarrow \text{id}$			$F \rightarrow (E)$		

递归下降语法分析器（或 LL 分析器）是基于表的解析器

预测分析表是一个二维数组，用于确定解析器在看到非终结符 A 和符号 a 时应该选择哪个产生式

LL(1)解析器的预测分析表没有包含多个产生式的条目

Parsing Table Construction

The following algorithm can be applied to any CFG

- **Input:** Grammar G **Output:** Parsing table M
- **Method:**
 - For each production $A \rightarrow \alpha$ of G , do the following:
 - For each terminal a in $\text{FIRST}(\alpha)$, add $A \rightarrow \alpha$ to $M[A, a]$
 - If ϵ is in $\text{FIRST}(\alpha)$, then for each terminal b (including the right endmarker $\$$) in $\text{FOLLOW}(A)$, add $A \rightarrow \alpha$ to $M[A, b]$
 - Set all empty entries in the table to **error**

构造预测分析表

以下算法可以应用于任何 CFG

输入：文法 G

输出：预测分析表 M

方法：对于每个产生式 $A \rightarrow \alpha$ ，进行以下处理：

执行以下操作：

- 对于 $\text{FIRST}(\alpha)$ 中每个终结符 a ，将 $A \rightarrow \alpha$ 加入到 $M[A, a]$
- 如果 ϵ 在 $\text{FIRST}(\alpha)$ 中，对于 $\text{FOLLOW}(A)$ 中每个终结符 b ，包括 $\$$ ，将 $A \rightarrow \alpha$ 加

入到 $M[A, b]$

将表中的所有空条目设置为错误

Parsing Table Construction Example

- Grammar**

- $E \rightarrow TE' \quad E' \rightarrow +TE' \mid \epsilon$
- $T \rightarrow FT' \quad T' \rightarrow *FT' \mid \epsilon \quad F \rightarrow (E) \mid \text{id}$

- **FIRST sets:** $E, T, F: \{ (, \text{id} \} \quad E': \{ +, \epsilon \} \quad T': \{ *, \epsilon \}$

- **FOLLOW sets:** $E, E': \{ \$,) \} \quad T, T': \{ +, \$,) \} \quad F: \{ *, +, \$,) \}$

NON - TERMINAL	INPUT SYMBOL					
	id	+	*	(	)	\$
E	$E \rightarrow TE'$			$E \rightarrow TE'$		
E'		$E' \rightarrow +TE'$			$E' \rightarrow \epsilon$	$E' \rightarrow \epsilon$
T	$T \rightarrow FT'$			$T \rightarrow FT'$		
T'		$T' \rightarrow \epsilon$	$T' \rightarrow *FT'$		$T' \rightarrow \epsilon$	$T' \rightarrow \epsilon$
F	$F \rightarrow \text{id}$			$F \rightarrow (E)$		

For $E \rightarrow TE'$:

FIRST(TE')
= FIRST(T)
= $\{ (, \text{id} \}$

Parsing Table Construction Example

- Grammar**

- $E \rightarrow TE' \quad E' \rightarrow +TE' \mid \epsilon$
- $T \rightarrow FT' \quad T' \rightarrow *FT' \mid \epsilon \quad F \rightarrow (E) \mid \text{id}$

- **FIRST sets:** $E, T, F: \{ (, \text{id} \} \quad E': \{ +, \epsilon \} \quad T': \{ *, \epsilon \}$

- **FOLLOW sets:** $E, E': \{ \$,) \} \quad T, T': \{ +, \$,) \} \quad F: \{ *, +, \$,) \}$

NON - TERMINAL	INPUT SYMBOL					
	id	+	*	(	)	\$
E	$E \rightarrow TE'$			$E \rightarrow TE'$		
E'		$E' \rightarrow +TE'$			$E' \rightarrow \epsilon$	$E' \rightarrow \epsilon$
T	$T \rightarrow FT'$			$T \rightarrow FT'$		
T'		$T' \rightarrow \epsilon$	$T' \rightarrow *FT'$		$T' \rightarrow \epsilon$	$T' \rightarrow \epsilon$
F	$F \rightarrow \text{id}$			$F \rightarrow (E)$		

For $E' \rightarrow \epsilon$:

ϵ in FIRST(ϵ)
FOLLOW(E')
= $\{ \$,) \}$

Conflicts in Parsing Tables

- **Grammar:** $S \rightarrow iEtSS' \mid a \quad S' \rightarrow eS \mid \epsilon \quad E \rightarrow b$

- $FIRST(eS) = \{e\}$, so we add $S' \rightarrow eS$ to $M[S', e]$
- $FOLLOW(S') = \{\$, e\}$, so we add $S' \rightarrow \epsilon$ to $M[S', e]$

NON - TERMINAL	INPUT SYMBOL					
	a	b	e	i	t	$\$$
S	$S \rightarrow a$			$S \rightarrow iEtSS'$		
S'			$S' \rightarrow \epsilon$ $S' \rightarrow eS$			$S' \rightarrow \epsilon$
E		$E \rightarrow b$				

- LL(1) grammar is never ambiguous.
- This grammar is not LL(1). The language has no LL(1) grammar !!!

预测分析表中的冲突

LL(1)文法不存在二义性

Conflicts in Parsing Tables

- **Grammar:** $S \rightarrow iEtSS' \mid a \quad S' \rightarrow eS \mid \epsilon \quad E \rightarrow b$

- $FIRST(eS) = \{e\}$, so we add $S' \rightarrow eS$ to $M[S', e]$
- $FOLLOW(S') = \{\$, e\}$, so we add $S' \rightarrow \epsilon$ to $M[S', e]$

NON - TERMINAL	INPUT SYMBOL					
	a	b	e	i	t	$\$$
S	$S \rightarrow a$			$S \rightarrow iEtSS'$		
S'			$S' \rightarrow \epsilon$ $S' \rightarrow eS$			$S' \rightarrow \epsilon$
E		$E \rightarrow b$				

Solution: Choose $S' \rightarrow eS$ to resolve ambiguity (associating an **else** with the closest previous **then**)

解决方案：选择 $S' \rightarrow es$ 解决歧义（将 else 与最近的一个 then 关联）

Recursive-Descent Parsing for LL(1) Grammars

```
void A() {  
1)   Choose an A-production,  $A \rightarrow X_1X_2 \cdots X_k$ ;  
2)   for (  $i = 1$  to  $k$  ) {  
3)       if (  $X_i$  is a nonterminal )  
4)           call procedure  $X_i()$ ;  
5)       else if (  $X_i$  equals the current input symbol  $a$  )  
6)           advance the input to the next symbol;  
7)       else /* an error has occurred */;  
   }  
}
```

Replace line 1 with: Choose A-production according to the parse table
• Assume input symbol is a , then the choice is the production in $M[A, a]$

LL(1)文法的递归下降语法分析

将第 1 行替换为：根据预测分析表选择 A-产生式

假设输入符号为 a , 那么选择 $M[A, a]$ 中的产生式

Non-Recursive Predictive Parsing

```
void A() {  
  1) Choose an A-production,  $A \rightarrow X_1 X_2 \cdots X_k$ ;  
  2) for (  $i = 1$  to  $k$  ) {  
  3)   if (  $X_i$  is a nonterminal )  
  4)   call procedure  $X_i()$ ;  
  5)   else if (  $X_i$  equals the current input symbol  $a$  )  
  6)     advance the input to the next symbol;  
  7)   else /* an error has occurred */;  
}
```



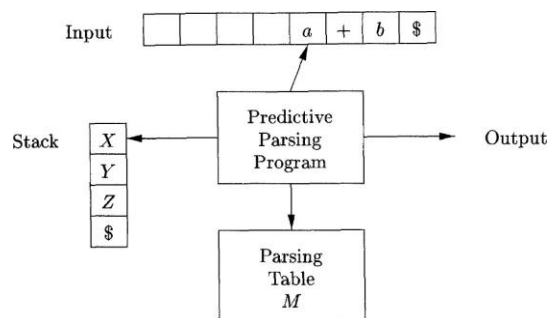
Recursive-descent parsing has recursive calls.
Can we design a non-recursive parser?

递归下降的语法分析具有递归调用。

我们能设计一个非递归解析器吗？

Non-Recursive Predictive Parsing

- A non-recursive predictive parser can be built by **explicitly maintaining a stack** (not implicitly via recursive calls)
 - **Input buffer** contains the string to be parsed, ending with \$
 - **Stack** holds a sequence of grammar symbols with \$ at the bottom. Initially, the stack contains only \$ and the start symbol S on top of \$



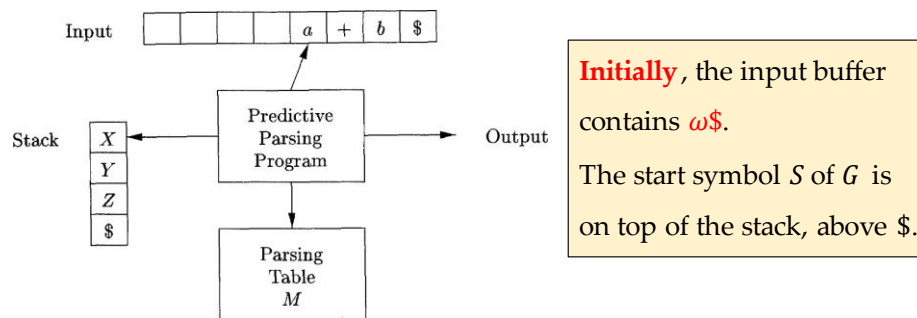
非递归的预测分析器可以通过显式维护堆栈（而不是通过递归调用隐式维护）来构建

- 输入缓冲区包含要解析的字符串，以\$结尾
- 堆栈包含底部为\$的语法符号序列。

最初，堆栈只包含\$和在\$顶部的开始符号 S

Table-Driven Predictive Parsing

- **Input:** A string ω and a parsing table M for grammar G
- **Output:** If ω is in $L(G)$, a leftmost derivation of ω ; otherwise, an error indication



表驱动的预测语法分析

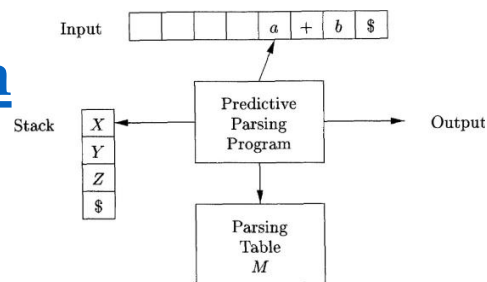
输入：字符串 ω 和文法 G 的预测分析表 M

输出：如果 ω 在 $L(G)$ 中，输出 ω 的一个最左边推导；否则，给出一个错误指示

最初，输入缓冲区包含 $\omega\$$

开始符号 S 位于堆栈顶部，在\$上方。

Parsing Algorithm



1. let a be the first symbol of ω ;
2. let X be the top stack symbol;
3. while ($X \neq \$$) { /* stack is not empty */
4. if ($X = a$) pop the stack and let a be the next symbol of ω ;
5. else if (X is a terminal) *error()*; /* X can only match a */
6. else if ($M[X, a]$ is an error entry) *error()*;
7. else if ($M[X, a] = X \rightarrow Y_1 Y_2 \dots Y_k$) {
8. output the production $X \rightarrow Y_1 Y_2 \dots Y_k$;
9. pop the stack;
10. push $Y_k, Y_{k-1}, \dots, Y_1$ onto the stack, with Y_1 on top; /* order is critical */
11. }
12. let X be the top stack symbol;
13. }

Example

$$\begin{aligned}
 E &\rightarrow TE' & E' &\rightarrow +TE' \mid \epsilon \\
 T &\rightarrow FT' & T' &\rightarrow *FT' \mid \epsilon & F &\rightarrow (E) \mid \text{id}
 \end{aligned}$$

NON - TERMINAL	INPUT SYMBOL					
	id	+	*	(	)	\$
E	$E \rightarrow TE'$			$E \rightarrow TE'$		
E'		$E' \rightarrow +TE'$			$E' \rightarrow \epsilon$	$E' \rightarrow \epsilon$
T	$T \rightarrow FT'$			$T \rightarrow FT'$		
T'		$T' \rightarrow \epsilon$	$T' \rightarrow *FT'$		$T' \rightarrow \epsilon$	$T' \rightarrow \epsilon$
F	$F \rightarrow \text{id}$			$F \rightarrow (E)$		

Input:
id + id * id

MATCHED	STACK	INPUT	ACTION
	$E\$$	id + id * id\$	
	$TE'\$$	id + id * id\$	output $E \rightarrow TE'$
	$FT'E'\$$	id + id * id\$	output $T \rightarrow FT'$
	id $T'E'\$$	id + id * id\$	output $F \rightarrow \text{id}$
	$T'E'\$$	+ id * id\$	match id
	$E'\$$	+ id * id\$	output $T' \rightarrow \epsilon$
	+ $TE'\$$	+ id * id\$	output $E' \rightarrow + TE'$
	$TE'\$$	id * id\$	match +

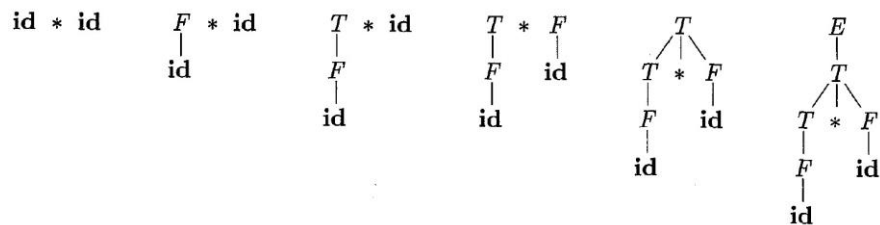
Matched part
+
Stack content
(from top to bottom)
=
A left-sentential form
总是最左句型

匹配部分+栈内容（从顶部到底部）=总是最左句型

Bottom-Up Parsing

$$\begin{array}{l} E \rightarrow E + T \mid T \\ T \rightarrow T * F \mid F \\ F \rightarrow (E) \mid \text{id} \end{array}$$

- **Problem definition:** Constructing a parse tree for an input string beginning at the leaves (**terminals**) and working up towards the root (**start symbol of the grammar**)
 - Equivalent to **finding a rightmost derivation (in reverse) for an input string**
- **Shift-reduce parsing (移入-归约分析技术)** is a **general style** of bottom-up parsing (using a **stack** to hold grammar symbols)



Fall 2024

DPPT Compilers

88

自底向上的语法分析

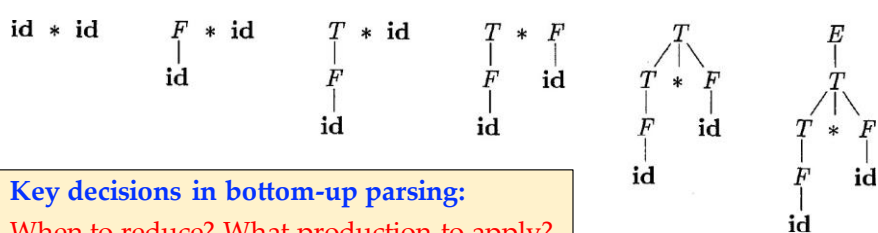
问题定义：为输入字符串构建一个从叶子（终结符）开始并向上延伸到根（语法的开始符号）的语法分析树

- 相当于为输入字符串找到最右边的推导（相反）

移入-归约分析技术是一种自底向上语法分析的通用方式（使用栈来保存语法符号）

Reductions (归约)

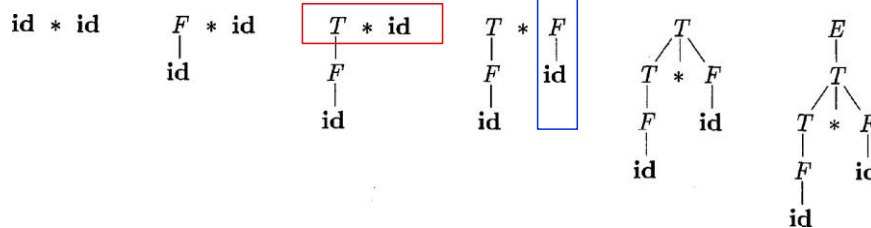
- Bottom-up parsing can be seen as a process of “reducing” a string ω to the start symbol of the grammar
- At each *reduction step*, a specific substring (at the top of the stack) matching the body of a production is replaced by the head of the production (the reverse of a step in derivation)



自底向上语法分析可以看作是一个将字符串 ω “归约”到开始符号的过程

在每一个归约步骤中，与产生式体匹配的特定子串（位于栈顶部）会被产生式的头部替换（与推导步骤相反）

Illustration of Challenges

$$\begin{aligned} E &\rightarrow E + T \mid T \\ T &\rightarrow T * F \mid F \\ F &\rightarrow (E) \mid \text{id} \end{aligned}$$


- For sentential form $T * \text{id}$, there are two applicable productions:
 - T is the body of $E \rightarrow T$
 - id is the body of $F \rightarrow \text{id}$
- Why reducing id to F , instead of reducing T to E ?
 - Reason:** If reducing T to E , $E * \text{id}$ is no longer a sentential form
 - Challenge:** how do we know this before making choices?



对于句型 $T * \text{id}$, 有两个适用的产生式:

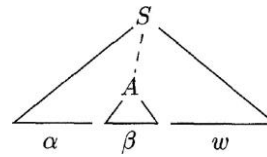
T 是 $E \rightarrow T$ 的体

id 是 $F \rightarrow \text{id}$ 的体

为什么把 id 归约为 F , 而不是将 T 归约为 E

- 原因: 如果将 T 归约为 E , $E * \text{id}$ 不再是句型
- 在这之前我们如何作决定?

Handles (句柄)



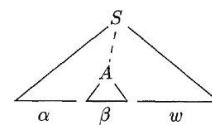
- Informally, a substring that matches the body of a production, and whose reduction represents one step along the reverse of a rightmost derivation
- Formally, if $S \xRightarrow{*}_{rm} \alpha A w \xRightarrow{}_{rm} \alpha \beta w$, then production $A \rightarrow \beta$ (or simply β) in the position following α is a handle of the sentential form $\alpha \beta w$

RIGHT SENTENTIAL FORM	HANDLE	REDUCING PRODUCTION
$\mathbf{id}_1 * \mathbf{id}_2$	$\mathbf{id}_1$	$F \rightarrow \mathbf{id}$
$F * \mathbf{id}_2$	F	$T \rightarrow F$
$T * \mathbf{id}_2$	$\mathbf{id}_2$	$F \rightarrow \mathbf{id}$
$T * F$	$T * F$	$E \rightarrow T * F$

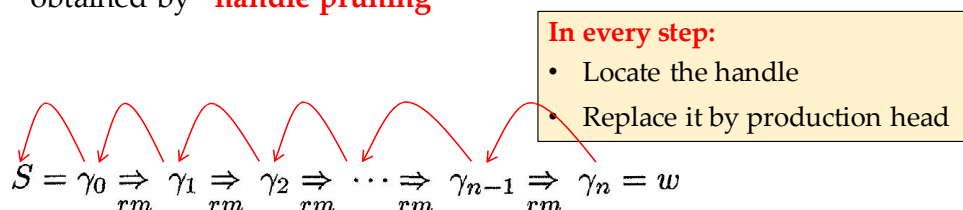
非正式地讲，句柄是和某个产生式体匹配的子串，对它的归约代表了相应的最右推导中的一个反向步骤

正式地讲，如果 $S \xRightarrow{*}_{rm} \alpha A w \xRightarrow{}_{rm} \alpha \beta w$ ，那么紧跟 α 的产生式 $A \rightarrow \beta$ （或者简单地 β ）是句型 $\alpha \beta w$ 的一个句柄

Handle Pruning (句柄剪枝)



- In a right-sentential form (最右句型), the string to the right of the handle must contain only terminal symbols
- If a grammar is unambiguous, every right-sentential form of the grammar has exactly one handle*
- A rightmost derivation in reverse (bottom-up parsing) can be obtained by “handle pruning”



* Given an unambiguous grammar, a right-sentential form can only be derived in one way

在最右句型中，句柄右侧的字符串必须仅包含终结符号

如果一个文法是无二义性的，那么该文法的每个右句型都有且只有一个句柄（如果文法二义，句柄可能不唯一）

通过句柄剪枝可以得到一个反向的最右推导

Shift-Reduce Parsing

- Shift-reduce parsing is a general style of bottom-up parsing in which:
 - A **stack** holds grammar symbols
 - An **input buffer** holds the rest of the string to be parsed
 - The **stack content (from bottom to top)** and **the input buffer content** form a **right-sentential form** (assuming no errors)

移入-归约语法分析技术是自底向上语法分析的一种通用方式，其中：

- 使用栈保存文法符号
- 使用输入缓冲区保存要进行语法分析的字符串的其余部分
- 栈内容（从底部到顶部）和输入缓冲区内容形成最右句型（假设没有错误）

Shift-Reduce Parsing

Initial status:

STACK	INPUT
\$	ω \$

Actions:

Shift
Reduce
Accept
Error

Shift-reduce process:

- The parser **shifts** zero or more input symbols onto the stack, until it is ready to reduce a string β (a **handle**) on top of the stack*
- **Reduce** β to the head of the appropriate production



The parser repeats the above cycle until it has detected an error or the stack contains the start symbol and input is empty

* During shift-reduce parsing, the handle will always eventually appear on top of the stack

初始状态:

移入-归约过程:

· 语法分析器将零个或多个输入符号转移到栈顶上, 直到它可以对栈顶的一个文法符号串 β 进行归约为止

- 将 β 归约为某个产生式的头

语法分析器重复上述循环, 直到检测到错误或栈包含开始符号且输入为空

*在移入-归约解析过程中, 句柄最终总是出现在栈的顶部

Example

Parsing steps on input $\text{id}_1 * \text{id}_2$

$E \rightarrow E + T \mid T$
 $T \rightarrow T * F \mid F$
 $F \rightarrow (E) \mid \text{id}$

STACK	INPUT	ACTION
\$	$\text{id}_1 * \text{id}_2$ \$	shift
\$ id_1	* id_2 \$	reduce by $F \rightarrow \text{id}$
\$ F	* id_2 \$	reduce by $T \rightarrow F$
\$ T	* id_2 \$	shift
\$ $T *$	id_2 \$	shift
\$ $T * \text{id}_2$	\$	reduce by $F \rightarrow \text{id}$
\$ $T * F$	\$	reduce by $T \rightarrow T * F$
\$ T	\$	reduce by $E \rightarrow T$
\$ E	\$	accept

Why Handles Always Appear at Stack Top?

(Analysis by enumeration) Consider the possible forms of two successive steps in any rightmost derivation*

$$(1) \quad S \xRightarrow{*}_{rm} \alpha A z \Rightarrow \alpha \beta B y z \Rightarrow \alpha \beta \gamma y z$$

- **Step 1:** the **rightmost nonterminal** A is replaced by **a string with nonterminals**. Suppose B is the rightmost nonterminal in the string.
- **Step 2:** B is replaced by a string γ

$$(2) \quad S \xRightarrow{*}_{rm} \alpha B x A z \Rightarrow \alpha B x y z \Rightarrow \alpha \gamma x y z$$

- **Step 1:** the **rightmost nonterminal** A is replaced by **a string y with terminals only**
- **Step 2:** the next rightmost nonterminal B (**somewhere to the left of y**) is replaced by a string γ

* The analysis is trivial when there is only one step of derivation

为什么句柄总是出现在栈顶?

(枚举分析) 考虑在任何最右推导中两个连续步骤的可能具有的形式

情况 1:

步骤 1: 最右边的非终结符 A 被一个带有非终结符的字符串替换。假设 B 是字符串中最右边的非终结符。

步骤 2: B 被替换为 γ

情况 2:

步骤 1: 最右边的非终结符 A 被替换为 y , y 中仅包含终结符。

步骤 2: 下一个最右边的非终结符 B 将位于 γ 左侧的某个地方

Why Handles Always Appear at Stack Top?

$$(1) \quad S \xRightarrow{*}_{rm} \alpha A z \Rightarrow \alpha \beta B y z \Rightarrow \alpha \beta \gamma y z$$

$$(2) \quad S \xRightarrow{*}_{rm} \alpha B x A z \Rightarrow \alpha B x y z \Rightarrow \alpha \gamma x y z$$

STACK	INPUT	ACTION
...	...	...
$\$ \alpha \beta \gamma$	$yz\$$	Reduce
$\$ \alpha \beta B$	$yz\$$	Shift
$\$ \alpha \beta B y$	$z\$$	Reduce
$\$ \alpha A$	$z\$$	Shift
...	...	...

x, y, z are strings of terminals

STACK	INPUT	ACTION
...	...	...
$\$ \alpha \gamma$	$xyz\$$	Reduce
$\$ \alpha B$	$xyz\$$	Shift
$\$ \alpha B x$	$yz\$$	Shift
$\$ \alpha B x y$	$z\$$	Reduce
$\$ \alpha B x A$	$z\$$	Shift
...	...	...

在这两种情况下, 语法分析器在进行一次归约后, 都必须接着移入零个或多个符号才
能在栈顶找到下一个句柄。因此它不需要到栈中间去寻找句柄。

Conflicts During Shift-Reduce Parsing

- There are context-free grammars for which shift-reduce parsing cannot be used
- Every shift-reduce parser for such a grammar can reach a configuration in which the parser, knowing the entire stack and also the next k input symbols, cannot decide
 - Whether to shift or to reduce (shift/reduce conflicts, 移入/归约冲突)
 - Which of several possible reductions to make (reduce/reduce conflicts, 归约/归约冲突)

移入-归约语法分析中的冲突

有些上下文无关文法不能使用移入-归约语法分析技术

这种文法的每一个移入-归约语法分析器都会达到这样一种格局：即使移入-归约语法分析器知道整个栈的内容以及下一个输入符号 k ，仍然无法决定是进行移入还是归约（移入/归约冲突）或者无法在多个可能的归约方法中选择正确的归约动作（归约/归约冲突）

Shift/Reduce Conflict Example

$$\begin{array}{lcl} stmt & \rightarrow & \text{if } expr \text{ then } stmt \\ & | & \text{if } expr \text{ then } stmt \text{ else } stmt \\ & | & \text{other} \end{array}$$

STACK
... if *expr* then *stmt*

INPUT
else ... \$

Reduce or shift? What if there is a *stmt* after *else*?



移入/归约冲突示例，是应该把栈内的进行规约还是继续移入？

Reduce/Reduce Conflict Example

- Parsing input **id(id, id)**

STACK	INPUT
\$id(id	, id\$

- (1) $stmt \rightarrow id (parameter_list)$
- (2) $stmt \rightarrow expr := expr$
- (3) $parameter_list \rightarrow parameter_list , parameter$
- (4) $parameter_list \rightarrow parameter$
- (5) $parameter \rightarrow id$
- (6) $expr \rightarrow id (expr_list)$
- (7) $expr \rightarrow id$
- (8) $expr_list \rightarrow expr_list , expr$
- (9) $expr_list \rightarrow expr$



Reduce by which production?

归约/归约冲突示例，不知用哪个产生式进行规约

LR Parsing (LR语法分析技术)

- **LR(k) parsers:** the most prevalent type of bottom-up parsers
 - **L:** left-to-right scan of the input
 - **R:** construct a rightmost derivation in reverse
 - **k :** use k input symbols of lookahead in making parsing decisions
- **LR(0) and LR(1) parsers** are of practical interest
 - When $k \geq 2$, the parser becomes too complex to construct (parsing table will be too huge to manage)

LR(k)语法分析器：最流行的自底向上的语法分析器类型

- **L:** 从左到右扫描输入
- **R:** 反向构造最右推导
- **k :** 在做出解析决策时使用 k 个前瞻性输入符号

LR(0)和 LR(1)语法分析器具有实际意义

当 $k \geq 2$ 时，语法分析器变得太复杂，无法构建（解析表太大，无法管理）

Advantages of LR Parsers

- **Table-driven** (like non-recursive LL parsers) and **powerful**
 - Although it is too much work to construct an LR parser by hand, there are parser generators to construct parsing tables automatically
 - Comparatively, LL parsers tend to be easier to write by hand, but less powerful (handle fewer grammars)
- LR-parsing is the most general **nonbacktracking shift-reduce parsing** method known
- LR parsers can be constructed to **recognize virtually all programming language constructs** for which CFGs can be written
- LR grammars can **describe more languages** than LL grammars
 - Recall the stringent conditions for a grammar to be LL(1)

LR 语法分析器的优势

- 表驱动（类似于非递归 LL 语法分析器）、强大
 - 尽管手工构建 LR 语法分析器工作量太大，但也有一些语法分析器生成器可以自动构建语法分析表
- 相比之下，LL 语法分析器往往更容易手工编写，但功能较弱（处理的语法较少）
- LR 语法分析技术是已知的最通用的非回溯移入-归约语法分析方法
- LR 语法分析器可以被构造为识别几乎所有可以为其编写 CFG 的编程语言结构
- LR 语法比 LL 语法可以描述更多的语言
 - 回想文法成为 LL(1)的严格条件

When to Shift/Reduce?

STACK	INPUT	ACTION
\$	id₁ * id₂ \$	shift
\$ id₁	* id₂ \$	reduce by $F \rightarrow \text{id}$
\$ F	* id₂ \$	reduce by $T \rightarrow F$
\$ T	* id₂ \$	shift
\$ T *	id₂ \$	shift
\$ T * id₂	\$	reduce by $F \rightarrow \text{id}$
\$ T * F	\$	reduce by $T \rightarrow T * F$
\$ T	\$	reduce by $E \rightarrow T$
\$ E	\$	accept

$E \rightarrow E + T \mid T$
 $T \rightarrow T * F \mid F$
 $F \rightarrow (E) \mid \text{id}$

Parsing input **id₁** * **id₂**

How does a shift/reduce parser know that T on stack top is not a handle (so the right action is to shift, not reducing T to E)?



移入-归约语法分析器如何知道 T 栈顶上没有句柄（所以正确的操作是移入，而不是归约 T 到 E ）?

LR(0) Items (LR(0) 项)

- An LR parser makes shift-reduce decisions by **maintaining states** to keep track of **what have been seen** in a parse
- An **LR(0) item** (item for short) is a **production with a dot** at some position of the body, indicating how much we have seen at a given point in the parsing process
 - $A \rightarrow \cdot XYZ$ $A \rightarrow X \cdot YZ$ $A \rightarrow XY \cdot Z$ $A \rightarrow XYZ \cdot$
 - $A \rightarrow X \cdot YZ$: we have just seen on the input a string derivable from X and we hope to see a string derivable from YZ next
- States**: sets of LR(0) items (**LR(0)项集**)

The production $A \rightarrow \epsilon$ generates only one item $A \rightarrow \cdot$

一个 LR 语法分析器通过维护一些状态来跟踪在语法分析中所处的位置，从而来做出移入-归约的决策。

LR(0)项（简称 item）是产生式再加上一个位于它的体中某处的点，表示在语法分析过程中的给定点上，我们已经看到了一个产生式的哪些部分。

$A \rightarrow \cdot XYZA \rightarrow X \cdot YZA \rightarrow XY \cdot ZA \rightarrow XYZ \cdot$

$A \rightarrow X \cdot YZ$: 我们刚刚在输入中看到了一个可以由 X 推导得到的字符串，我们希望接下来看到一个能从 YZ 推导得到的字符串

状态：LR(0)项集

产生式 $A \rightarrow \epsilon$ 仅生成一个 item $A \rightarrow \cdot$

Canonical LR(0) Collection

- One collection of sets of LR(0) items, called the **canonical LR(0) collection** (规范LR(0)项集族), provides the basis for constructing a DFA that is used to make parsing decisions
- To construct canonical LR(0) collection for a grammar, we need to define:
 - An augmented grammar (增广文法)
 - Two functions:
 - (1) CLOSURE of item sets (项集闭包)
 - (2) GOTO

规范 LR(0)项集族

LR(0)项集合的一个集合，称为规范 LR(0)项集族，为构建用于做出语法分析决定的 DFA 提供了基础

为了构造文法的 LR(0)项集规范族，我们需要定义：

- 增广文法
- 两个函数：(1) 项集闭包的 CLOSURE 和 (2) GOTO

Augmented Grammar

- Augmenting a grammar G with start symbol S
 - Introduce a new start symbol S' to replace S
 - Add a new production $S' \rightarrow S$
- Obviously, $L(G) = L(G')$
- **Benefit:** With the augmentation, acceptance occurs when and only when the parser is about to reduce by $S' \rightarrow S$
 - Otherwise, acceptance could occur at many points since there may be multiple S -productions

增广文法

增广一个以 S 为开始符号的文法 G :

- 引入一个新开始符号 S' 代替 S
- 增加产生式 $S' \rightarrow S$

明显地 $L(G) = L(G')$

优点：通过增广，只有当语法分析器使用 $S' \rightarrow S$ 进行归约时输入符号串被接受。否则，输入符号串可能在多处被接受，因为可能存在多个 S -产生式

Closure of Item Sets

- If I is a set of items for a grammar G , then $\text{CLOSURE}(I)$ is the set of items constructed from I by the two rules
 1. Initially, add every item in I to $\text{CLOSURE}(I)$
 2. If $A \rightarrow \alpha \cdot B\beta$ is in $\text{CLOSURE}(I)$ and $B \rightarrow \gamma$ is a production, then add the item $B \rightarrow \cdot \gamma$ to $\text{CLOSURE}(I)$, if it is not already there. Apply this rule until no more new items can be added to $\text{CLOSURE}(I)$
- **Intuition:** $A \rightarrow \alpha \cdot B\beta$ indicates that we hope to see a substring derivable from $B\beta$. This substring will have a prefix derivable from B . Therefore, we add items for all B -productions.

项集闭包

如果 I 是文法 G 的一组项集, 那么 $\text{CLOSURE}(I)$ 就是按照下面两条规则构造的项集:

1. 一开始, 将 I 中的各个项加入到 $\text{CLOSURE}(I)$
2. 如果 $A \rightarrow \alpha \cdot B\beta$ 在 $\text{CLOSURE}(I)$ 中, $B \rightarrow \gamma$ 是一个产生式, 并且 $B \rightarrow \cdot \gamma$ 不在 $\text{CLOSURE}(I)$ 中, 那么将 $B \rightarrow \cdot \gamma$ 加入 $\text{CLOSURE}(I)$ 中。不断应用这条规则, 直到没有新项可以加入到

直观地讲: $A \rightarrow \alpha \cdot B\beta$ 表示我们希望看到从一个能够从 $B\beta$ 推导得到的子串。这个子串的某个前缀可以从 B 推导得到。因此, 我们将所有 B -产生式加入到项集。

Algorithm for CLOSURE(I)

// the earlier natural language description is already clear enough

```
SetOfItems CLOSURE( $I$ ) {  
     $J = I$ ;  
    repeat  
        for ( each item  $A \rightarrow \alpha \cdot B \beta$  in  $J$  )  
            for ( each production  $B \rightarrow \gamma$  of  $G$  )  
                if (  $B \rightarrow \cdot \gamma$  is not in  $J$  )  
                    add  $B \rightarrow \cdot \gamma$  to  $J$ ;  
    until no more items are added to  $J$  on one round;  
    return  $J$ ;  
}
```

Example

$E \rightarrow E + T \mid T$
$T \rightarrow T * F \mid F$
$F \rightarrow (E) \mid \text{id}$

- Augmented grammar

▪ $E' \rightarrow E$ $E \rightarrow E + T \mid T$ $T \rightarrow T * F \mid F$ $F \rightarrow (E) \mid \text{id}$

- Computing the closure of the item set $\{[E' \rightarrow \cdot E]\}$

- Initially, $[E' \rightarrow \cdot E]$ is in the closure
- Add $[E \rightarrow \cdot E + T]$ and $[E \rightarrow \cdot T]$ to the closure
- Add $[T \rightarrow \cdot T * F]$ and $[T \rightarrow \cdot F]$ to the closure
- Add $[F \rightarrow \cdot (E)]$ and $[F \rightarrow \cdot \text{id}]$ to the closure (reached fixed point)

The Function GOTO

$E \rightarrow E + T \mid T$ $T \rightarrow T * F \mid F$ $F \rightarrow (E) \mid \text{id}$
--

- **GOTO(I, X)**, where I is a set of items and X is a grammar symbol, is defined to be the closure of the set of all items $[A \rightarrow \alpha X \cdot \beta]$ such that $[A \rightarrow \alpha \cdot X \beta]$ is in I
 - $\text{CLOSURE}(\{[A \rightarrow \alpha X \cdot \beta] \mid [A \rightarrow \alpha \cdot X \beta] \in I\})$
- **Example:** Computing $\text{GOTO}(I, +)$ for $I = \{[E' \rightarrow E \cdot], [E \rightarrow E \cdot + T]\}$
 - There is only one item $[E \rightarrow E \cdot + T]$ in which $+$ follows $\cdot$
 - Then compute the $\text{CLOSURE}(\{[E \rightarrow E \cdot + T]\})$, which contains
 - $[E \rightarrow E + \cdot T]$
 - $[T \rightarrow \cdot T * F] \downarrow [T \rightarrow \cdot F]$
 - $[F \rightarrow \cdot (E)] \downarrow [F \rightarrow \cdot \text{id}]$

GOTO 函数

返回项目集 I 对应于文法符号 X 的后继项目集闭包

Constructing Canonical LR(0) Collection

```

void items( $G'$ ) {
     $C = \{\text{CLOSURE}(\{[S' \rightarrow \cdot S]\})\};$   $\longrightarrow$  Initial item set
    repeat
        for ( each set of items  $I$  in  $C$  )
            for ( each grammar symbol  $X$  )
                if (  $\text{GOTO}(I, X)$  is not empty and not in  $C$  )
                    add  $\text{GOTO}(I, X)$  to  $C$ ;
    until no new sets of items are added to  $C$  on a round;
}

```

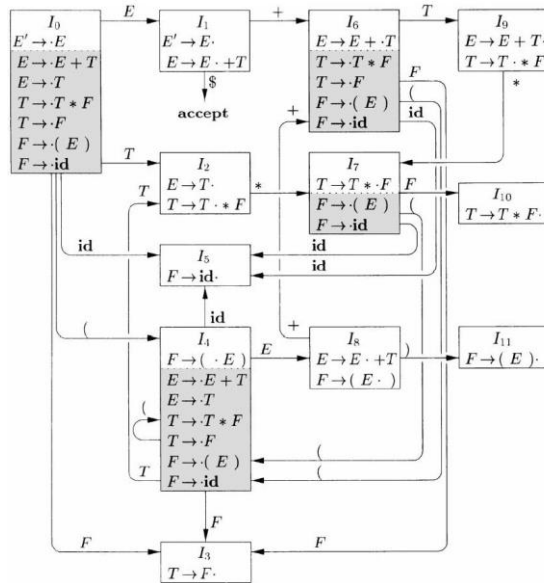
Iteratively find all possible GOTO targets (corresponding to states in the automaton for parsing)

构造规范 LR(0)项集族

迭代查找所有可能的 GOTO 目标（对应于用于语法分析的自动机中的状态）

Example

The canonical LR(0) collection for the grammar below is $\{I_0, I_1, \dots, I_{11}\}$



- (1) $E \rightarrow E + T$
- (2) $E \rightarrow T$
- (3) $T \rightarrow T * F$
- (4) $T \rightarrow F$
- (5) $F \rightarrow (E)$
- (6) $F \rightarrow id$

LR(0) Automaton

- The central idea behind “Simple LR”, or SLR, is constructing the LR(0) automaton from the grammar
 - The states are the item sets in the canonical LR(0) collection
 - The transitions are given by the GOTO function
 - The start state is $CLOSURE(\{S' \rightarrow \cdot S\})$

LR(0)自动机

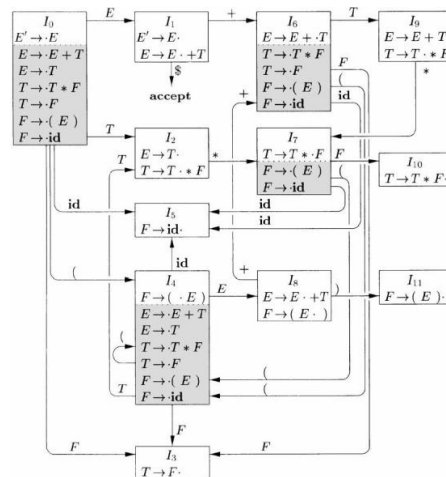
“简单 LR 语法分析技术”（即 SLR 分析技术）的中心思想是根据文法构造出 LR(0)自动机。

- 这个自动机的状态是规范 LR(0)项集族
- 它的转换由 GOTO 函数给出
- 开始状态是 $CLOSURE(\{S' \rightarrow \cdot S\})$

The Use of LR(0) Automaton

Helps make shift-reduce decisions:

- Suppose that the string γ of **grammar symbols** takes the automaton from the start **state 0** to some **state j**
- **Shift** on next **input symbol a** if **state j** has a **transition on a**
- Otherwise, **reduce**; the items in state $j will tell us which production to use$



LR(0)自动机的使用

帮助做出移入-归约决定:

- 假设文法符号 γ 使自动机从开始状态 0 运行到某个状态 j
- 如果下一个输入符号为 a 且状态 j 有一个在 a 上的转换, 那就移入 a
- 否则, 归约; 状态 j 的项将会告诉我们使用哪个产生式进行归约

Example: Parsing **id * id** (1)

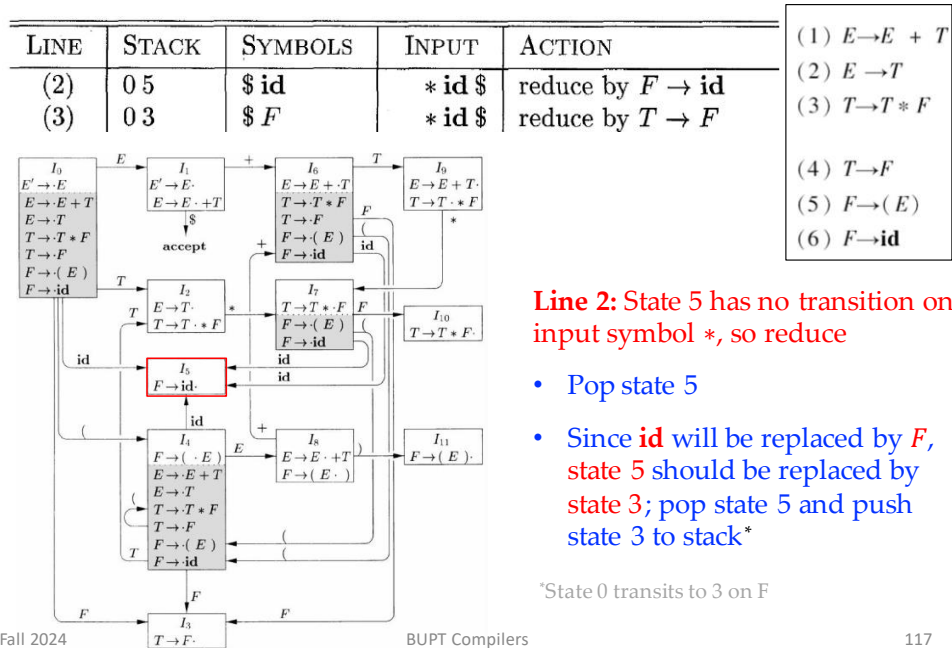
- We only keep states in the stack; grammar symbols can be recovered from the states

LINE	STACK	SYMBOLS	INPUT	ACTION
(1)	0	\$	id * id \$	shift to 5
(2)	0 5	\$ id	* id \$	reduce by $F \rightarrow \text{id}$
(3)	0 3	\$ F	* id \$	reduce by $T \rightarrow F$
(4)	0 2	\$ T	* id \$	shift to 7
(5)	0 2 7	\$ T *	id \$	shift to 5
(6)	0 2 7 5	\$ T * id	\$	reduce by $F \rightarrow \text{id}$
(7)	0 2 7 10	\$ T * F	\$	reduce by $T \rightarrow T * F$
(8)	0 2	\$ T	\$	reduce by $E \rightarrow T$
(9)	0 1	\$ E	\$	accept

栈只保存状态，文法符号可以从相应状态中获取

从状态 0 开始，移入 id，进入状态 5，id 出栈规约为 F，F 进栈，状态 5 出栈，状态 3 进栈；

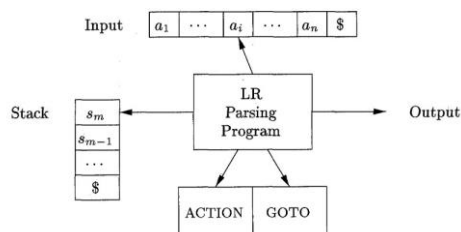
Example: Parsing $\text{id} * \text{id}$ (2)



在第 2 行，状态 5 已压入栈中，从状态 5 出发没有输入 $*$ 上的转换，因此选择归约。
因为 id 将会被 F 代替，状态 5 将会被状态 3 代替，所以弹出状态 5，压入状态 3。

LR Parser Structure

- An LR parser consists of an **input**, an **output**, a **stack**, a **driver program**, and a **parsing table** (ACTION + GOTO)
- The driver program is the same for all LR parsers; only the parsing table changes from one parser to another
- The stack holds a sequence of states
 - In SLR, the stack holds states from the LR(0) automaton
- The parser decides the next action based on (1) the state at the top of the stack and (2) the terminal read from the input buffer



LR 语法分析器结构

LR 语法分析器由一个输入、一个输出、一个栈、一个驱动程序和一个语法分析表 (ACTION+GOTO) 组成

所有 LR 语法分析器的驱动程序都是相同的；只有语法分析表随语法分析器的不同而变化

栈保存一个状态序列

在 SLR 中，栈保存来自 LR(0)自动机的状态

语法分析器根据 (1) 栈顶部的状态和 (2) 从输入缓冲区读取的终结符来决定下一个动作

Parsing Table: ACTION + GOTO

- The **ACTION** function takes two arguments: (1) a state i and (2) a terminal a (or \$)
- **ACTION** $[i, a]$ can have one of the four forms of values:
 - **Shift j** : shift input a to the stack, but uses state j to represent a
 - **Reduce $A \rightarrow \beta$** : reduce β on the top of the stack to head A
 - **Accept**: The parser accepts the input and finishes parsing
 - **Error**: syntax errors exist
- The **GOTO** function is extended from the one defined on sets of items to states: if $\text{GOTO}(\bar{l}_i, A) = \bar{l}_j$, then $\text{GOTO}(i, A) = j$

LR 语法分析表: ACTION + GOTO

ACTION 函数接受两个参数: (1) 状态 i 和 (2) 终结符 a (或\$)

ACTION $[i, a]$ 可以具有以下四种形式的值之一:

- 移入 j : 其中 j 是一个状态, 把输入符号 a 移入栈中, 但使用状态 j 代表 a
- 归约 $A \rightarrow \beta$: 将栈顶的 β 归约为产生式头 A
- 接受: 语法分析器接受输入并完成语法分析过程

- 报错：存在语法错误

将定义在项集上的 GOTO 函数扩展为定义在状态集上的函数：如果 $\text{GOTO}(I_i, A) = I_j$ ，那么 GOTO 也把状态 i 和一个非终结符 A 映射到状态 j

LR Parser Configurations (态势)

- “**Configuration**” is notation for representing the complete state of the parser. A *configuration* is a pair:

Stack contents (top on the right) $\leftarrow (s_0 s_1 \dots s_m, a_i a_{i+1} \dots a_n \$) \rightarrow$ Remaining input

- By construction, each state (except s_0) in an LR parser corresponds to a set of items and a grammar symbol (the symbol that leads to the state transition, i.e., the symbol on the incoming edges)
 - Suppose X_i is the grammar symbol for state s_i
 - Then $X_0 X_1 \dots X_m a_i a_{i+1} \dots a_n$ is a **right-sentential form** (assume no errors)

LR 语法分析器的态势

“态势”是表示语法分析器完整状态的符号。态势是一个形如： $(s_0 s_1 \dots s_m, a_i a_{i+1} \dots a_n \$)$ 的对

第一个分量是栈中的内容，第二个分量是余下的输入。

根据结构，每个状态（除 s_0 ）对应于一组项和文法符号（导致状态转换的符号，即传入边上的符号）

- 假设 X_i 是状态 s_i 的文法符号
- 那么 $X_0 X_1 \dots X_m a_i a_{i+1} \dots a_n$ 是最右句型（假设没有错误）

Behavior of the LR Parser

- For the configuration $(s_0 s_1 \dots s_m, a_i a_{i+1} \dots a_n \$)$, the LR parser checks $ACTION[s_m, a_i]$ in the parsing table to decide the parsing action
 - **shift s** : shift the next state s onto the stack, entering the configuration $(s_0 s_1 \dots s_m s, a_{i+1} \dots a_n \$)$
 - **reduce $A \rightarrow \beta$** : execute a reduce move, entering the configuration $(s_0 s_1 \dots s_{m-r} s, a_i a_{i+1} \dots a_n \$)$, where r = the length of β , and $s = GOTO(s_{m-r}, A)$
 - **accept**: parsing is completed
 - **error**: the parser has found an error and calls an error recovery routine

LR-Parsing Algorithm

- **Input**: The parsing table for a grammar G and an input string ω
- **Output**: If ω is in $L(G)$, the reduction steps of a bottom-up parse for ω ; otherwise, an error indication
- **Initial configuration**: $(s_0, \omega \$)$

```
let  $a$  be the first symbol of  $w\$$ ;  
while(1) { /* repeat forever */  
    let  $s$  be the state on top of the stack;  
    if (  $ACTION[s, a] = \text{shift } t$  ) {  
        push  $t$  onto the stack;  
        let  $a$  be the next input symbol;  
    } else if (  $ACTION[s, a] = \text{reduce } A \rightarrow \beta$  ) {  
        pop  $|\beta|$  symbols off the stack;  
        let state  $t$  now be on top of the stack;  
        push  $GOTO[t, A]$  onto the stack;  
        output the production  $A \rightarrow \beta$ ;  
    } else if (  $ACTION[s, a] = \text{accept}$  ) break; /* parsing is done */  
    else call error-recovery routine;  
}
```

Parsing Table Example

STATE	ACTION						GOTO		
	id	+	*	(	)	\$	E	T	F
0	s5			s4			1	2	3
1		s6				acc			
2		r2	s7		r2	r2			
3		r4	r4		r4	r4			
4	s5			s4			8	2	3
5		r6	r6		r6	r6			
6	s5			s4				9	3
7	s5			s4					10
8		s6			s11				
9		r1	s7		r1	r1			
10		r3	r3		r3	r3			
11		r5	r5		r5	r5			

- (1) $E \rightarrow E + T$
- (2) $E \rightarrow T$
- (3) $T \rightarrow T * F$
- (4) $T \rightarrow F$
- (5) $F \rightarrow (E)$
- (6) $F \rightarrow id$

- s5: shift by pushing state 5 r3: reduce using production No. 3
- GOTO entries for terminals are not listed, can be checked in ACTION part

LR Parsing Example

- (1) $E \rightarrow E + T$
- (2) $E \rightarrow T$
- (3) $T \rightarrow T * F$
- (4) $T \rightarrow F$
- (5) $F \rightarrow (E)$
- (6) $F \rightarrow id$

- Input string: **id * id + id**

	STACK	SYMBOLS	INPUT	ACTION
(1)	0		id * id + id \$	shift
(2)	0 5	id	* id + id \$	reduce by $F \rightarrow id$
(3)	0 3	F	* id + id \$	reduce by $T \rightarrow F$
(4)	0 2	T	* id + id \$	shift
(5)	0 2 7	T *	id + id \$	shift
(6)	0 2 7 5	T * id	+ id \$	reduce by $F \rightarrow id$
(7)	0 2 7 10	T * F	+ id \$	reduce by $T \rightarrow T * F$
(8)	0 2	T	+ id \$	reduce by $E \rightarrow T$
(9)	0 1	E	+ id \$	shift
(10)	0 1 6	E +	id \$	shift
(11)	0 1 6 5	E + id	\$	reduce by $F \rightarrow id$
(12)	0 1 6 3	E + F	\$	reduce by $T \rightarrow F$
(13)	0 1 6 9	E + T	\$	reduce by $E \rightarrow E + T$
(14)	0 1	E	\$	accept

- Push state 5
- Remove id
- Pop three states 2, 7, 10
- Push state 2 (GOTO[0, T] = 2)

Constructing SLR-Parsing Tables (1)

- The SLR-parsing table for a grammar G can be constructed based on the LR(0) item sets and LR(0) automaton
 1. Construct the canonical LR(0) collection $\{I_0, I_1, \dots, I_n\}$ for the augmented grammar G'
 2. State i is constructed from I_i . ACTION can be determined as follows:
 - If $[A \rightarrow \alpha \cdot a\beta]$ is in I_i and $\text{GOTO}[I_i, a] = I_j$, then set $\text{ACTION}[i, a]$ to “shift j ” (here a must be a terminal)
 - If $[A \rightarrow \alpha \cdot]$ is in I_i , then set $\text{ACTION}[i, a]$ to “reduce $A \rightarrow \alpha$ ” for **all a in FOLLOW(A)**; here A may not be S'
 - If $[S' \rightarrow S \cdot]$ is in I_i , then set $\text{ACTION}[i, \$]$ to “accept”
 3. The goto transitions for state i are constructed for all nonterminals A using the rule: If $\text{GOTO}(I_i, A) = I_j$, then $\text{GOTO}(i, A) = j$

构造 SLR 语法分析表

SLR 语法分析表 G 以 LR(0) 项和 LR(0) 自动机为基础。

1. 构造增广文法 G' 的规范 LR(0) 项集族 $\{I_0, I_1, \dots, I_n\}$
2. 根据 I_i 构造得到状态 i ，状态 i 的语法分析动作按照下面的方法决定：
 - 如果 $[A \rightarrow \alpha \cdot a\beta]$ 在 I_i 中并且 $\text{GOTO}[I_i, a] = I_j$ ，那么将 $\text{ACTION}[i, a]$ 设置为“移入 j ”
 - 如果 $[A \rightarrow \alpha \cdot]$ 在 I_i 中，那么对于 **FOLLOW(A)** 中的所有 a ，将设置 $\text{ACTION}[i, a]$ 为“归约 $A \rightarrow \alpha$ ”。这里 A 不等于 S'
 - 如果 $[S' \rightarrow S \cdot]$ 在 I_i 中，那么将设置 $\text{ACTION}[i, \$]$ 为“接受”
3. 状态 i 对于各个非终结符号 A 的转换使用下面的规则构造得到：如果 $\text{GOTO}[I_i, A] = I_j$ ，那么 $\text{GOTO}[i, A] = j$

Constructing SLR-Parsing Tables (2)

4. All entries not defined in steps 2 and 3 are set to “**error**”
5. Initial state is the one constructed from the item set containing $[S' \rightarrow \cdot S]$

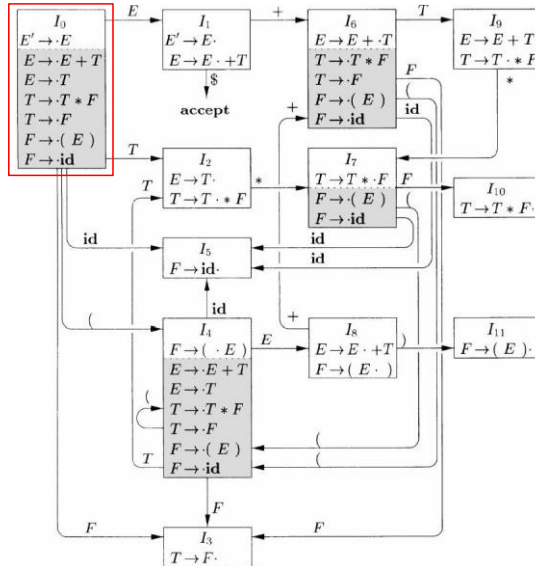
If there is no conflict during the parsing table construction (i.e., multiple actions for a table entry), the grammar is **SLR(1)**

4. 规则 2、3 没有定义的所有条目都设置为 “报错”
5. 语法分析器的初始状态就是根据 $[S' \rightarrow \cdot S]$ 所在项集构造得到的状态

如果在语法分析构造过程中没有冲突（即，一个表项的多个条目），则语法为 SLR

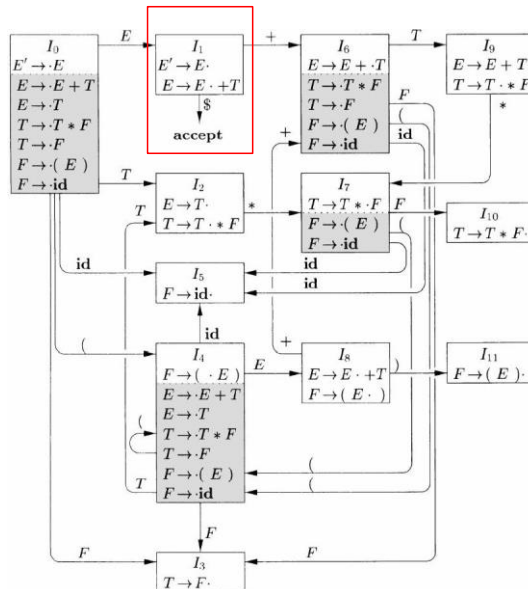
(1)

Example (1)



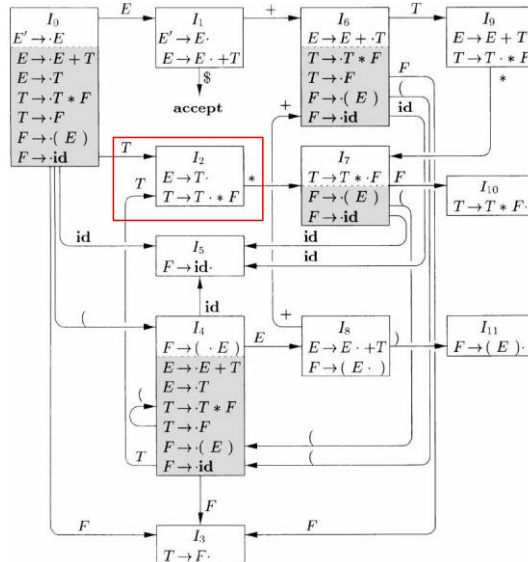
- $\text{ACTION}(0, \text{ }) = s4$ (shift 4)
- $\text{ACTION}(0, \text{id}) = s5$
- $\text{GOTO}[0, E] = 1$
- $\text{GOTO}[0, T] = 2$
- $\text{GOTO}[0, F] = 3$

Example (2)



- $\text{ACTION}(1, +) = s6$
- $\text{ACTION}(1, \$) = \text{accept}$

Example (3)



- $\text{ACTION}(2, *) = s7$
- $\text{ACTION}(2, \$) = \text{reduce } E \rightarrow T$
- $\text{ACTION}(2, +) = \text{reduce } E \rightarrow T$
- $\text{ACTION}(2,) = \text{reduce } E \rightarrow T$

$\text{FOLLOW}(E) = \{\$, +,)\}$

Non-SLR Grammar

- Grammar
 - $S \rightarrow L = R \mid R$
 - $L \rightarrow * R \mid \text{id}$
 - $R \rightarrow L$
- For item set I_2 :
 - According to item #1:
 $\text{ACTION}[2, =]$ is "s6"
 - According to item #2:
 $\text{ACTION}[2, =]$ is "reduce $R \rightarrow L$ "
($\text{FOLLOW}(R)$ contains =)

$I_0: S' \rightarrow \cdot S$	$I_5: L \rightarrow \text{id} \cdot$
$S \rightarrow \cdot L = R$	
$S \rightarrow \cdot R$	$I_6: S \rightarrow L = \cdot R$
$L \rightarrow \cdot * R$	$R \rightarrow \cdot L$
$L \rightarrow \cdot \text{id}$	$L \rightarrow \cdot * R$
$R \rightarrow \cdot L$	$L \rightarrow \cdot \text{id}$
$I_1: S' \rightarrow S \cdot$	$I_7: L \rightarrow * R \cdot$
$I_2: S \rightarrow L \cdot = R$	$I_8: R \rightarrow L \cdot$
$R \rightarrow L \cdot$	$I_9: S \rightarrow L = R \cdot$
$I_3: S \rightarrow R \cdot$	
$I_4: L \rightarrow * \cdot R$	
$R \rightarrow \cdot L$	
$L \rightarrow \cdot * R$	
$L \rightarrow \cdot \text{id}$	

This grammar is not ambiguous

CLR and LALR will succeed on a larger collection of grammars, including the above one. However, there exist unambiguous grammars for which every LR parser construction method will encounter conflicts.

CLR 和 LALR 将在更大的文法集合上取得成功，包括上面的文法集合。然而，存在着明确的文法，对于这些文法，每个 LR 语法分析器构造方法都会遇到冲突。

Weakness of the SLR Method

- In SLR, the state i calls for reduction by $A \rightarrow \alpha$ if the item set I_i contains item $[A \rightarrow \alpha \cdot]$ and input symbol a is in $\text{FOLLOW}(A)$
- In some situations, after reduction, **the content $\beta\alpha$ on stack top would become βA that cannot be followed by a in any right-sentential form**

在 SLR 中，状态 i 要求归约 $A \rightarrow \alpha$ ，如果项集 I_i 包含项 $[A \rightarrow \alpha \cdot]$ ，同时输入符号 a 在 $\text{FOLLOW}(A)$ 中

在某些情况下，归约后 $\beta\alpha$ 在栈顶上会变成 βA ，后面跟着的不是 a 的任何最右句型

Example: Parsing $\text{id} = \text{id}$

- $S \rightarrow L = R \mid R$
- $L \rightarrow * R \mid \text{id}$
- $R \rightarrow L$

I_0 : $S' \rightarrow \cdot S$
 $S \rightarrow \cdot L = R$
 $S \rightarrow \cdot R$
 $L \rightarrow \cdot * R$
 $L \rightarrow \cdot \text{id}$
 $R \rightarrow \cdot L$

I_1 : $S' \rightarrow S \cdot$

I_2 : $S \rightarrow L \cdot = R$
 $R \rightarrow L \cdot$

I_3 : $S \rightarrow R \cdot$

I_4 : $L \rightarrow * \cdot R$
 $R \rightarrow \cdot L$
 $L \rightarrow \cdot * R$
 $L \rightarrow \cdot \text{id}$

I_5 : $L \rightarrow \text{id} \cdot$

I_6 : $S \rightarrow L = \cdot R$
 $R \rightarrow \cdot L$
 $L \rightarrow \cdot * R$
 $L \rightarrow \cdot \text{id}$

I_7 : $L \rightarrow * R \cdot$

I_8 : $R \rightarrow L \cdot$

I_9 : $S \rightarrow L = R \cdot$

Stack	Symbols	Input	Action
\$0		id = id	Shift 5
\$05	id	= id	Reduce $L \rightarrow \text{id}$
\$02	L	= id	Suppose reduce $R \rightarrow L$
\$03	R	= id	Error!

* Cannot shift, cannot reduce since $\text{FOLLOW}(S) = \{\$ \}$

Problem: SLR reduce too casually

How to know if a reduction is a good move?
 Utilize the next input symbol to precisely
 determine whether to call for a reduction.

如何知道归约是否是一个好的决定？利用下一个输入符号精确地确定是否要求归约。

LR(1) Item

- **Idea:** Carry more information in the state to rule out some invalid reductions (**splitting LR(0) states**)
- General form of an LR(1) item: $[A \rightarrow \alpha \cdot \beta, a]$
 - $A \rightarrow \alpha\beta$ is a production and a is a terminal or $\$$
 - “1” refers to the length of the 2nd component: the **lookahead** (向前看字符)*
 - The lookahead symbol has no effect **if β is not ϵ** since it only helps determine whether to reduce (a will be inherited during state transitions)
 - An item of the form $[A \rightarrow \alpha \cdot, a]$ calls for a reduction by $A \rightarrow \alpha$ **only if the next input symbol is a** (the set of such a 's is a **subset** of $\text{FOLLOW}(A)$)

*: LR(0) items do not have lookahead symbols, and hence they are called LR(0)

规范 LR(1)项

在状态中携带更多信息以排除一些无效的归约（通过分裂 LR (0) 状态）

LR(1)项的一般形式: $[A \rightarrow \alpha \cdot \beta, a]$

- $A \rightarrow \alpha\beta$ 是一个产生式, a 是一个终结符或右端结束标记
- 1 指的是第二个分量 a 的长度, 第二个分量 a 叫做向前看字符
- 当 β 不为空时, 向前看字符没有任何作用因为它决定是否归约
- 一个形如 $[A \rightarrow \alpha \cdot, a]$ 的项只有在下一个输入符号等于 a 时, 才会按照 $A \rightarrow \alpha$ 进行归约。这样的 a 的集合总是的 FOLLOW(A) 子集

$A \rightarrow \alpha\beta$ 是一个产生式和 a 是终端还是\$ “1” 是指第二个组成部分的长度: 展望(向前看字符)*如果 β 不是 ϵ 因为它只帮助确定是否减少(a 将在状态转换期间继承) 表单中的一
项 $A \rightarrow \alpha \cdot, a$ 要求减少 $A \rightarrow \alpha$ 仅当下一个输入符号 a (a' s 是 FOLLOW 的子集(A))*:

LR (0) 项没有先行符号, 因此它们被称为 LR (0)

Constructing LR(1) Item Sets (1)

- Constructing the collection of LR(1) item sets is essentially the same as constructing the canonical collection of LR(0) item sets. The only differences lie in the CLOSURE and GOTO functions.

```
SetOfItems CLOSURE(I) {  
    J = I;  
    repeat  
        for ( each item  $[A \rightarrow \alpha \cdot B \beta, a]$  in I )  
            for ( each production  $B \rightarrow \gamma$  in  $G'$  )  
                for ( each terminal  $b$  in FIRST( $\beta a$ ) )  
                    add  $[B \rightarrow \gamma, b]$  to set J;  
    until no more items are added to J;  
    return J;  
}
```

```
SetOfItems CLOSURE(I) {  
    J = I;  
    repeat  
        for ( each item  $A \rightarrow \alpha \cdot B \beta$  in J )  
            for ( each production  $B \rightarrow \gamma$  of G )  
                if (  $B \rightarrow \gamma$  is not in J )  
                    add  $B \rightarrow \gamma$  to J;  
    until no more items are added to J on one round;  
    return J;  
}
```

It only generates the new item $[B \rightarrow \gamma, b]$ from $[A \rightarrow \alpha \cdot B \beta, a]$ if b is in FIRST(βa)

构造 LR (1) 项集族本质上与构造规范 LR (0) 项集族相同。唯一的区别在于 CLOSURE 和 GOTO 函数。

Why b should be in $\text{FIRST}(\beta a)$?

- **An informal analysis:**

- The lookahead b decides when to reduce γ to B
- Suppose the algorithm also generates an item for b that is NOT in $\text{FIRST}(\beta a)$
- At a certain point, when we see γ on stack top and b as next input, we would reduce γ to B ($[B \rightarrow \gamma \cdot, b]$ must come from $[B \rightarrow \cdot \gamma, b]$)
- Since we generate $[B \rightarrow \cdot \gamma, b]$ from $[A \rightarrow \alpha \cdot B \beta, a]$ during closure computation, we hope to see $\alpha B \beta$ at stack top sometime after reducing γ to B and reduce using $A \rightarrow \alpha B \beta$ when a is the next input symbol
- Unfortunately, this is impossible when b is not in $\text{FIRST}(\beta a)$
- **Then why generate $[B \rightarrow \cdot \gamma, b]$ from $[A \rightarrow \alpha \cdot B \beta, a]$???**

Constructing LR(1) Item Sets (2)

- Constructing the collection of LR(1) item sets is essentially the same as constructing the canonical collection of LR(0) item sets. The only differences lie in the CLOSURE and **GOTO** functions.

```
SetOfItems GOTO( $I, X$ ) {  
    initialize  $J$  to be the empty set;  
    for ( each item  $[A \rightarrow \alpha \cdot X \beta, a]$  in  $I$  )  
        add item  $[A \rightarrow \alpha X \cdot \beta, a]$  to set  $J$ ;  
    return CLOSURE( $J$ );  
}
```

GOTO(I, X) in LR(0) item sets:

The closure of the set of all items $[A \rightarrow \alpha X \cdot \beta]$ such that $[A \rightarrow \alpha \cdot X \beta]$ is in I .

The lookahead symbols are passed to new items from existing items

构造 LR (1) 项集族本质上与构造规范 LR (0) 项集族相同。唯一的区别在于 CLOSURE 和 GOTO 函数。

向前看符号从现有项传递到新项

Constructing LR(1) Item Sets (3)

```
void items( $G'$ ) {  
     $C = \{\text{CLOSURE}(\{[S' \rightarrow \cdot S]\})\};$   
    repeat  
        for ( each set of items  $I$  in  $C$  )  
            for ( each grammar symbol  $X$  )  
                if ( GOTO( $I, X$ ) is not empty and not in  $C$  )  
                    add GOTO( $I, X$ ) to  $C$ ;  
    until no new sets of items are added to  $C$  on a round;  
}
```

Constructing the
collection of LR(0)
item sets

```
void items( $G'$ ) {  
    initialize  $C$  to  $\{\text{CLOSURE}(\{[S' \rightarrow \cdot S, \$]\})\};$   
    repeat  
        for ( each set of items  $I$  in  $C$  )  
            for ( each grammar symbol  $X$  )  
                if ( GOTO( $I, X$ ) is not empty and not in  $C$  )  
                    add GOTO( $I, X$ ) to  $C$ ;  
    until no new sets of items are added to  $C$ ;  
}
```

Constructing the
collection of LR(1)
item sets



LR(1) Item Sets Example

- Augmented grammar:

- $S' \rightarrow S \quad S \rightarrow CC \quad C \rightarrow cC \mid d$

- Constructing I_1 item set and GOTO function:

- $I_0 = \text{CLOSURE}([S' \rightarrow \cdot S, \$]) =$

- $\{[S' \rightarrow \cdot S, \$], [S \rightarrow \cdot CC, \$], [C \rightarrow \cdot cC, c/d], [C \rightarrow \cdot d, c/d]\}$

$\text{FIRST}(\$) = \{\$ \}$

$\text{FIRST}(C\$) = \{c, d\}$

- $\text{GOTO}(I_0, S) = \text{CLOSURE}(\{[S' \rightarrow S \cdot, \$]\}) = \{[S' \rightarrow S \cdot, \$]\}$

- $\text{GOTO}(I_0, C) = \text{CLOSURE}(\{[S \rightarrow C \cdot C, \$]\}) =$

- $\{[S \rightarrow C \cdot C, \$], [C \rightarrow \cdot cC, \$], [C \rightarrow \cdot d, \$]\}$

$\text{FIRST}(\$) = \{\$ \}$

- $\text{GOTO}(I_0, c) = \text{CLOSURE}(\{[C \rightarrow c \cdot C, c/d]\}) =$

- $\{[C \rightarrow c \cdot C, c/d], [C \rightarrow \cdot cC, c/d], [C \rightarrow \cdot d, c/d]\}$

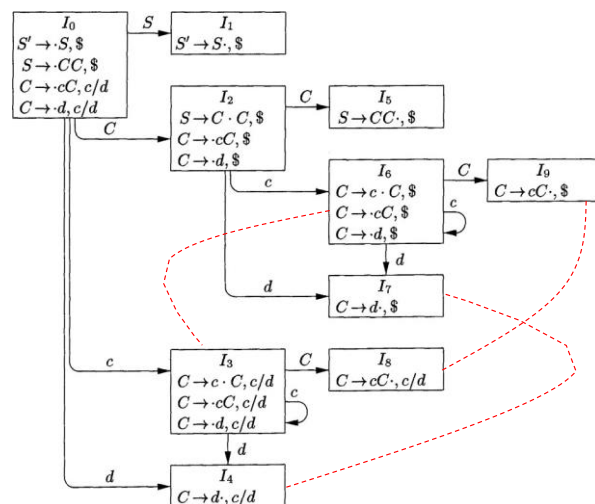
- $\text{GOTO}(I_0, d) = \text{CLOSURE}(\{[C \rightarrow d \cdot, c/d]\}) = \{[C \rightarrow d \cdot, c/d]\}$

The GOTO Graph Example

10 states in total

These states are equivalent if we ignore the lookahead symbols (SLR makes no such distinctions of states):

- I_3 and I_6
- I_4 and I_7
- I_8 and I_9



Constructing Canonical LR(1) Parsing Tables (1)

1. Construct $C' = \{I_0, I_1, \dots, I_n\}$, the collection of LR(1) item sets for the augmented grammar G'
2. State i of the parser is constructed from I_i . Its parsing action is determined as follows:
 - If $[A \rightarrow \alpha \cdot a\beta, b]$ is in I_i and $\text{GOTO}(I_i, a) = I_j$, then set $\text{ACTION}[i, a]$ to “*shift j*.” Here, a must be a terminal.
 - If $[A \rightarrow \alpha \cdot, a]$ is in I_i , $A \neq S'$, then set $\text{ACTION}[i, a]$ to “*reduce $A \rightarrow \alpha$* ”
 - If $[S' \rightarrow S \cdot, \$]$ is in I_i , then set $\text{ACTION}[i, \$]$ to “*accept*”

If any **conflicting actions** result from the above rules, we say the grammar is **not LR(1)**

构造规范 LR(1)语法分析表

1. 构造增广文法 G' 的规范 LR(1)项集族 $\{I_0, I_1, \dots, I_n\}$

2. 根据 I_i 构造得到状态 i , 状态 i 的语法分析动作按照下面的方法决定:

- 如果 $[A \rightarrow \alpha \cdot a\beta, b]$ 在 I_i 中并且 $\text{GOTO}[I_i, a] = I_j$, 那么将 $\text{ACTION}[i, a]$ 设置为 “移入 j ”
- 如果 $[A \rightarrow \alpha \cdot, a]$ 在 I_i 中, 设置 $\text{ACTION}[i, a]$ 为 “归约 $A \rightarrow \alpha$ ”。这里 A 不等于 S'
- 如果 $[S' \rightarrow S \cdot, \$]$ 在 I_i 中, 那么将设置 $\text{ACTION}[i, \$]$ 为 “接受”

如果以上规则导致任何冲突的动作, 我们就说这个文法不是 LR(1)的。

Constructing Canonical LR(1) Parsing Tables (2)

3. The goto transitions for state i are constructed from all nonterminals A using the rule: If $\text{GOTO}(I_i, A) = I_j$, then $\text{GOTO}(i, A) = j$
4. All entries not defined in steps (2) and (3) are made “error”
5. The initial state of the parser is the one constructed from the set of items containing $[S' \rightarrow \cdot S, \$]$

3. 状态 i 对于各个非终结符号 A 的转换使用下面的规则构造得到：如果 $\text{GOTO}[I_i, A] = I_j$ ，那么 $\text{GOTO}[i, A] = j$
4. 规则 2、3 没有定义的所有条目都设置为“报错”
5. 语法分析器的初始状态就是根据 $[S' \rightarrow \cdot S, \$]$ 所在项集构造得到的状态

LR(1) Parsing Table Example

- **Grammar:**

- $S' \rightarrow S$
- $S \rightarrow CC$
- $C \rightarrow cC \mid d$

- These pairs of states can be seen as being split from the corresponding LR(0) states:

- (3, 6)
- (4, 7)
- (8, 9)

STATE	ACTION			GOTO	
	<i>c</i>	<i>d</i>	\$	<i>S</i>	<i>C</i>
0	s3	s4		1	2
1			acc		
2	s6	s7			5
3	s3	s4			8
4	r3	r3			
5			r1		
6	s6	s7			9
7			r3		
8	r2	r2			
9			r2		

Error Recovery in LR Parsing

- An LR parser should be able to handle errors:
 - Report the precise location of an error
 - Recover from an error and continue with the parsing
- Two typical error recovery strategies
 - Panic-mode recovery (恐慌模式)
 - Phrase-level recovery (短语层次的恢复)

LR 语法分析器应该能够处理错误：

- 报告错误的精确位置
- 从错误中恢复并继续分析

两种典型的错误

- 恐慌模式
- 短语层次的恢复

Panic-Mode Recovery

- **Basic idea:** Discard zero or more input symbols until a synchronization token (同步词法单元) is found
- **Rationale:**
 - The parser always looks for a prefix of the input that can be derivable from a non-terminal A
 - When there is an error, it means it is impossible to find such a prefix
 - If errors only occur in the part related to A , we can skip the part by looking for a symbol that can legitimately follow A
 - **Example:** If A is *stmt*, then the synchronization symbol can be a semicolon

恐慌模式

基本思想：丢弃零个或多个输入符号，直到同步词法单元已找到

原理：

- 语法分析器总是查找可以从非终结符 A 推导的输入的前缀
- 当出现错误时，意味着无法找到这样的前缀
- 如果错误仅发生在与 A 相关的部分，我们可以通过寻找一个可以合法跟随的符号

来跳过这一部分

- 示例：如果 A 是 $stmt$ ，则同步词法单元可以是分号

Phrase-Level Recovery

- **Basic idea:**

- Examine each error entry in the parsing table and decide the most likely programmer error that would give rise to the error
- Modify the top of the stack or first input symbols and issue messages to programmers

STATE	ACTION						GOTO
	id	+	*	(	)	\$	
0	s3	e1	e1	s2	e2	e1	1
1	e3	s4	s5	e3	e2	acc	
2	s3	e1	e1	s2	e2	e1	6
3	r4	r4	r4	r4	r4	r4	
4	s3	e1	e1	s2	e2	e1	7
5	s3	e1	e1	s2	e2	e1	8
6	e3	s4	s5	e3	s9	e4	
7	r1	r1	s5	r1	r1	r1	
8	r2	r2	r2	r2	r2	r2	
9	r3	r3	r3	r3	r3	r3	

Example phrase-level recovery:

- Remove the right) from the input;
- Issue diagnostic “unbalanced right parenthesis”.

短语层次的恢复

基本思路：

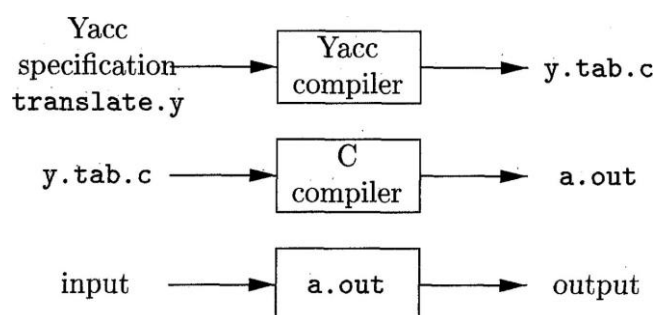
- 检查语法分析表中的每个错误条目，并确定最有可能导致错误的程序员错误
- 修改栈顶部或第一个输入符号，并向程序员发出消息

短语层次的恢复示例：

- 删除右侧的)
- 提示“不匹配的右括号”。

The Parser Generator YACC/BISON

- Yacc: yet another compiler-compiler
- Bison: an extension and improvement of Yacc



本节将介绍如何使用语法分析器生成工具来帮助构造一个编译器的前端，我们将使用 LALR 语法分析生成工具 Yacc 作为讨论的基础。Yacc，又一个编译器的编译器

Yacc 在 UNIX 系统中是以命令的方式出现的，它已经用于实现多个编译器产品。Bison 是 GNU 项目的一部分，是对 Yacc 的扩展和改进。

Structure of YACC Source Programs

- **Declarations (声明)**

- Ordinary C declarations
- Grammar tokens

- **Translation rules (翻译规则)**

- Rule = a grammar production + the associated semantic action

- **Supporting C routines (辅助性C语言例程)**

- Directly copied to `y.tab.c`
- Can be invoked in the semantic actions
- `yylex()` must be provided, which returns tokens
- Other procedures such as error recovery routines may be provided

declarations %% translation rules %% supporting C routines
--

声明

- 通常的 C 声明
- 对词法单元的声明

翻译规则

- 翻译规则 = 产生式 + 语义动作

辅助性 C 语言例程

- 将会被直接拷贝至 `y.tab.c`
- 可以在语义动作中调用
- 必须提供 `yylex()` 这一词法分析器，它将返回一个由词法单元和相关属性组成的

词法单元

- 可以添加错误恢复例程等其他过程

Translation Rules

$$\begin{array}{lcl} \langle \text{head} \rangle & : & \langle \text{body} \rangle_1 \quad \{ \langle \text{semantic action} \rangle_1 \} \\ & | & \langle \text{body} \rangle_2 \quad \{ \langle \text{semantic action} \rangle_2 \} \\ & & \dots \\ & | & \langle \text{body} \rangle_n \quad \{ \langle \text{semantic action} \rangle_n \} \\ & ; & \end{array}$$

- The first head is taken to be the **start symbol**
- A semantic action is a sequence of C statements
 - $\$ \$$ refers to the attribute value associated with the nonterminal of the head
 - $\$ i$ refers to the value associated with i th grammar symbol of the body
- A semantic action is performed when we reduce by the associated production
 - We can compute a value for $\$ \$$ in terms of the $\$ i$'s

翻译规则

第一个产生式头符号被视为开始符号

一个语义动作是一个 C 语句的序列

- $\$ \$$ 表示和相应产生式头部的非终结符关联的属性值
- $\$ i$ 表示和相应产生式体中第 i 个文法符号关联的属性值

当我们按照一个产生式归约时就会执行该产生式相关联的语义动作

- 我们通常根据 $\$ i$ 的值来计算 $\$ \$$ 的值

YACC Source Program Example

```
%{
#include <ctype.h>
%}

%token DIGIT

%%
line  : expr '\n'      { printf("%d\n", $1); }
      ;
expr  : expr '+' term  { $$ = $1 + $3; }
      | term
      ;
term  : term '*' factor { $$ = $1 * $3; }
      | factor
      ;
factor: '(' expr ')'   { $$ = $2; }
      | DIGIT
      ;

%%
yylex() {
    int c;
    c = getchar();
    if (isdigit(c)) {
        yylval = c-'0';
        return DIGIT;
    }
    return c;
}
```

Fall 2024

BUPT Compilers

152

Conflicts Resolution in YACC

- **Default strategy:**
 - **Shift/reduce conflicts:** always shift
 - **Reduce/reduce conflicts:** reduce with the production listed first
- **Specifying the precedence and associativity of terminals**
 - **Associativity:** %left, %right, %nonassoc
 - **Shift a /reduce $A \rightarrow a$ conflict:** compare the precedence of a and $A \rightarrow a$ (use associativity when precedence is not enough)
 - The declaration order of terminals determine their precedence
 - The precedence of a production is equal to the precedence its rightmost terminal. It can also be specified using %prec<terminal> which defines the precedence of the production to be the same as the terminal

Fall 2024

BUPT Compilers

153

Yacc 的冲突解决

默认策略：

- 移入/归约冲突：始终移入
- 归约/归约冲突：归约第一个列出的产生式

指定终结符的优先级和结合性：

- 结合性：%left、%right、%nonassoc
- 移入 a /归约 $A \rightarrow a$ 冲突：比较的优先级 a 和 $A \rightarrow a$ （优先级不够时使用结合性）

性）

- 终结符的声明顺序决定了它们的优先级
- 一个产生式的优先级等于它最右边终结符的优先级。也可以使用

用%prec<terminal>指定，它将产生式的优先级定义为与终结符相同

Error Recovery in YACC

- In YACC, error recovery uses a form of error productions
 - General form: $A \rightarrow \text{error } \alpha$
 - The users can decide which nonterminals (e.g., those generating expressions, statements, blocks, etc.) will have error productions
- Example: $\text{stmt} \rightarrow \text{error } ;$
 - When the parser encounters an error, it would skip just beyond the next semicolon and assumes that a statement had been found
 - The semantic action of the error production will be invoked: it would not need manipulate the input, but could simply generate a diagnostic message

Yacc 的错误恢复

在 Yacc 中，错误恢复使用一种错误产生式的形式：

- 一般形式： $A \rightarrow \text{error } \alpha$
- 用户可以决定哪些非终结符（例如，生成表达式、语句、块等的非终结符）将

具有错误产生式

错误示例: ***stmt*** $\rightarrow$ **error**;

- 该错误产生式规定当语法分析器遇到错误时, 它要跳到下一个分号之后, 并假设已经找到一条语句
- 将调用错误产生式的语义动作不需要处理输入, 只需生成诊断消息